

Name, Binding and Scope

Name, binding and scope

- **Name**

- A name is a mnemonic character string used to represent variables, constants, operations, types, function and so on using symbolic identifiers rather than low-level concepts like addresses.
- Names are also essential in the context of a second meaning of the word abstraction. Eg:
 - Function name define what function is going to do rather than how it do.
 - Subroutines are control abstractions: they allow the programmer to hide arbitrarily complicated code behind a simple interface.
 - Classes are data abstractions: they allow the programmer to hide data representation details behind a (comparatively) simple set of operations.
 - x, sin, f, prog1 are names.
 - 1, 2, 3, "test" are not names.
 - +, <=, ... may be names if they are not built-in operations.

Name, binding and scope

- **Binding**

- A binding is an association between two things, such as a name and the thing it names.
- Bindings active at a certain point in a program called binding time.
- Binding time is the time at which a binding is created or, more generally, the time at which any implementation decision is made.
 - The scope of binding is the region of a program or time interval(s) in the program's execution during which the binding is active or A maximal region of the program where no bindings are destroyed (e.g., a function body)
 - The binding time depend on the execution environment or reference environment..

Name, binding and scope

- OBJECT AND BINDING LIFETIME

- Object lifetime

- Period between the creation and destruction of the object.

- Binding lifetime

- Period of time between the creation and the destruction of a name-to object binding is called the binding's lifetime.
 - These above lifetimes need not necessarily coincide. In particular, an object may retain its value, even when a given name can no longer be used to access it.

Two common mistakes:

- **Dangling reference:** no object for a binding

(E.g., a pointer refers to an object that has already been deleted)

- **Memory leak:** No binding for an object (Prevents the object from being deallocated)

Name, binding and scope

- Compile time:
 - Map high-level language constructs to machine code
 - Layout static data in memory
- Link time:
 - Since most compilers support separate compilation—compiling different modules of a program at different times.
 - A program is usually not complete until the various modules are joined together by a linker. The linker chooses the overall layout of the modules with respect to one another, and resolves inter-module references.
 - When a name in one module refers to an object in another module, the binding between the two is not finalized until link time.

Name, binding and scope

- Load time:
 - Load time refers to the point at which the operating system loads the program into memory so that it can run.
 - The choice of machine addresses for objects within the program was not finalized until load time.
 - Most modern operating systems distinguish between virtual and physical addresses. Virtual addresses are chosen at link time; physical addresses can actually change at run time. The processor's memory management hardware translates virtual addresses into physical addresses during each individual instruction at run time
- Run time:
 - Bindings of values to variables occur at run time, as the decisions vary from language to language.
 - Allocate dynamic data and assign it to variables

Name, binding and scope

- IMPORTANCE OF BINDING TIMES

- Early binding (compile time, link time, load time):
 - Faster code
 - Typical in compiled languages
- Late binding (runtime):
 - Greater flexibility
 - Typical in interpreted languages

Name, binding and scope

- STORAGE MANAGEMENT OF LIFETIME

- Object lifetimes generally correspond to one of three principal storage allocation mechanisms:
 - Static objects are given an absolute address that is retained throughout the program's execution.
 - Stack objects are allocated and deallocated in last-in, first-out order, usually in conjunction with subroutine calls and returns.
 - Heap objects may be allocated and deallocated at arbitrary times. They require a more general (and expensive) storage management algorithm.

Name, binding and scope

- STATIC ALLOCATION

- Global variables are the obvious example of static allocation.
- Variables that are local to a single subroutine, but retain their values from one invocation to the next; their space is statically allocated.
- Numeric and string-valued constant literals are also statically allocated, for statements such as $A = B/14.7$.
- Variety of tables that are used by runtime eg: debugging, dynamic-type checking, garbage collection, exception handling, and other purposes; these are also statically allocated.
- Languages that do not support recursion compiler may choose to use static allocation for local variables, effectively arranging for the variables of different invocations to share the same locations, and thereby avoiding any run-time overhead for creation and destruction. E.g., early versions of FORTRAN.

Name, binding and scope

- STACK ALLOCATION

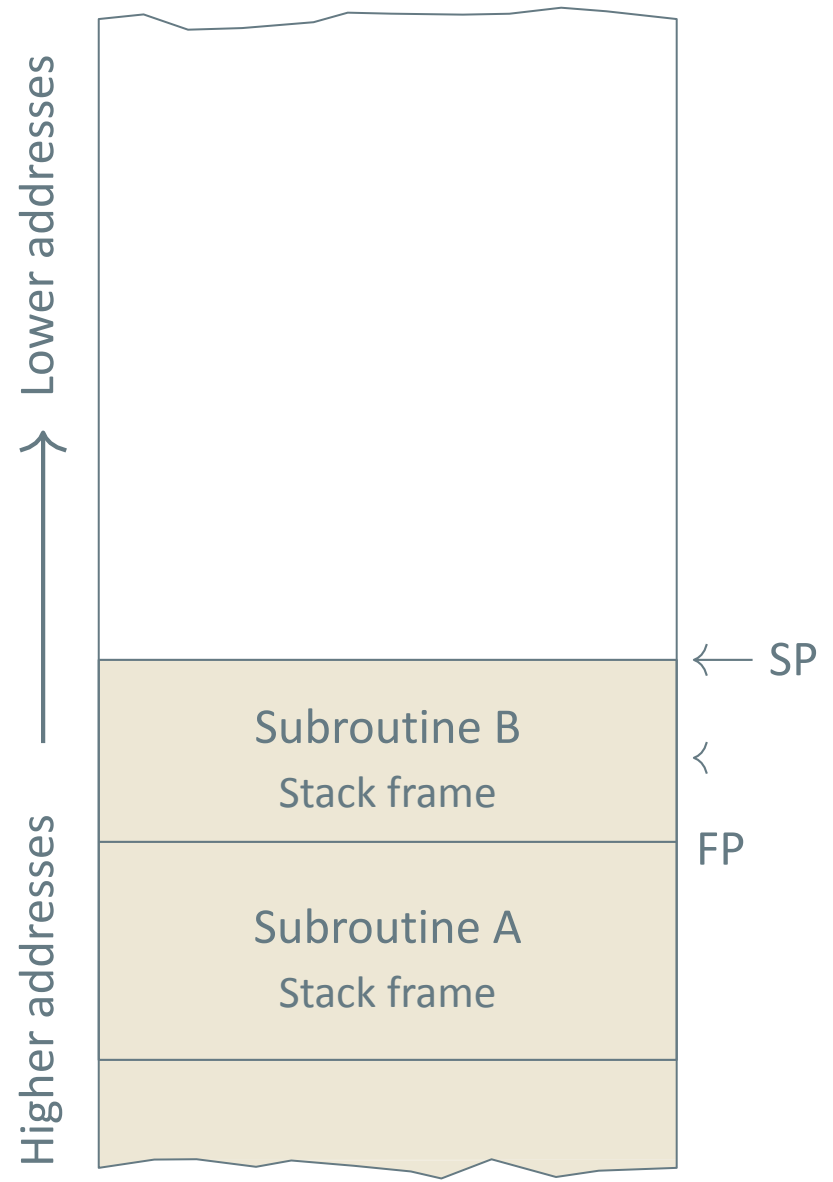
- If a language permits recursion, static allocation of local variables is no longer an option, since the number of instances of a variable that may need to exist at the same time is conceptually unbounded.
- Nesting of subroutine calls cause allocate space for locals on a stack.
- Each instance of a subroutine at run time has its own frame (also called an activation record) on the stack, containing:
 - Arguments and return values : compilers keep these in registers.
 - local variables, temporaries: usually intermediate values. Again, a good compiler will keep them in registers
 - Bookkeeping information: This may include the subroutine's return address, a reference to the stack frame of the caller etc.

Name, binding and scope

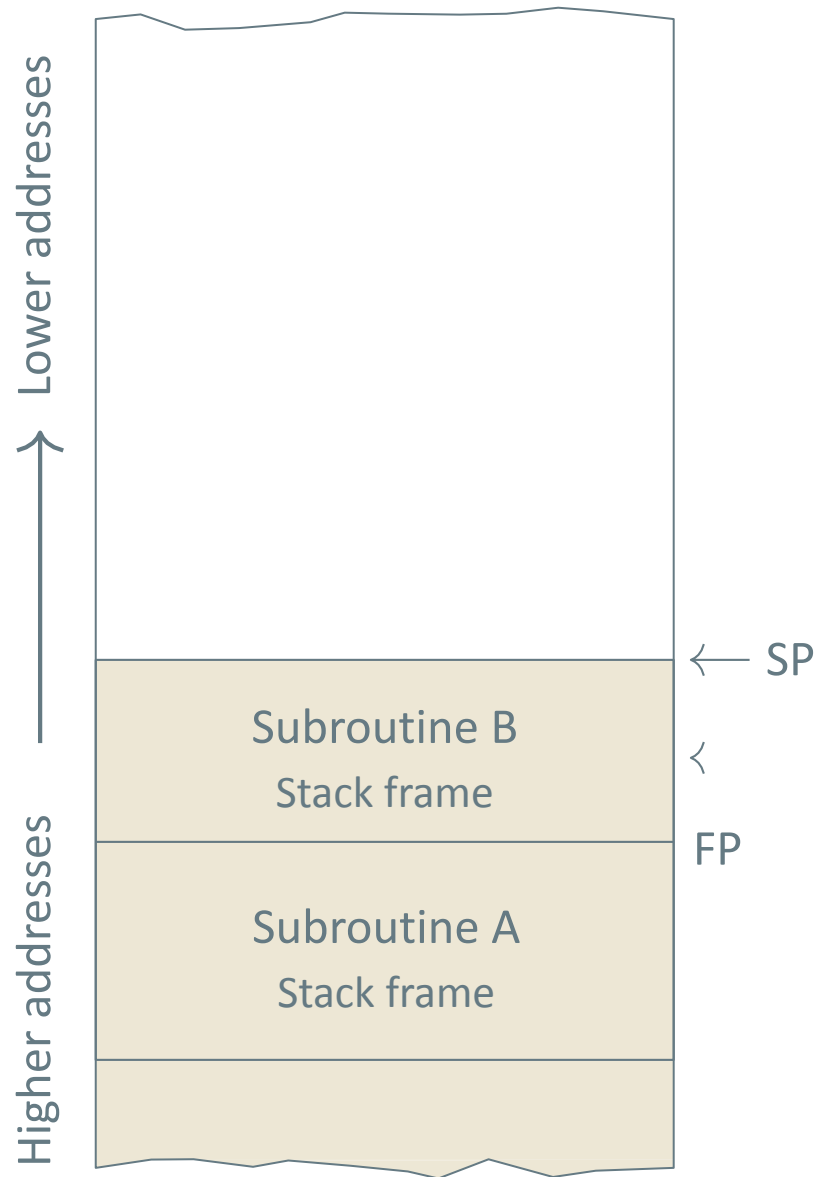
- STACK ALLOCATION

- Maintenance of the stack is the responsibility of the subroutine calling sequence.
 - Before call, the caller pushes arguments and return address onto the stack.
 - After being called (prologue-code executed at the beginning), the callee initializes local variables, etc.
 - Before returning (epilogue-code executed at the end), the callee cleans up local data.
 - After the call returns, the caller retrieves return value(s) and restores the stack to its state before the call.
- The runtime management of the stack involves two registers:
 - Frame pointer: Points to a known location within the current stack frame
 - Stack pointer: Points to the first unused location on the stack (as a starting position for the next stack frame)

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

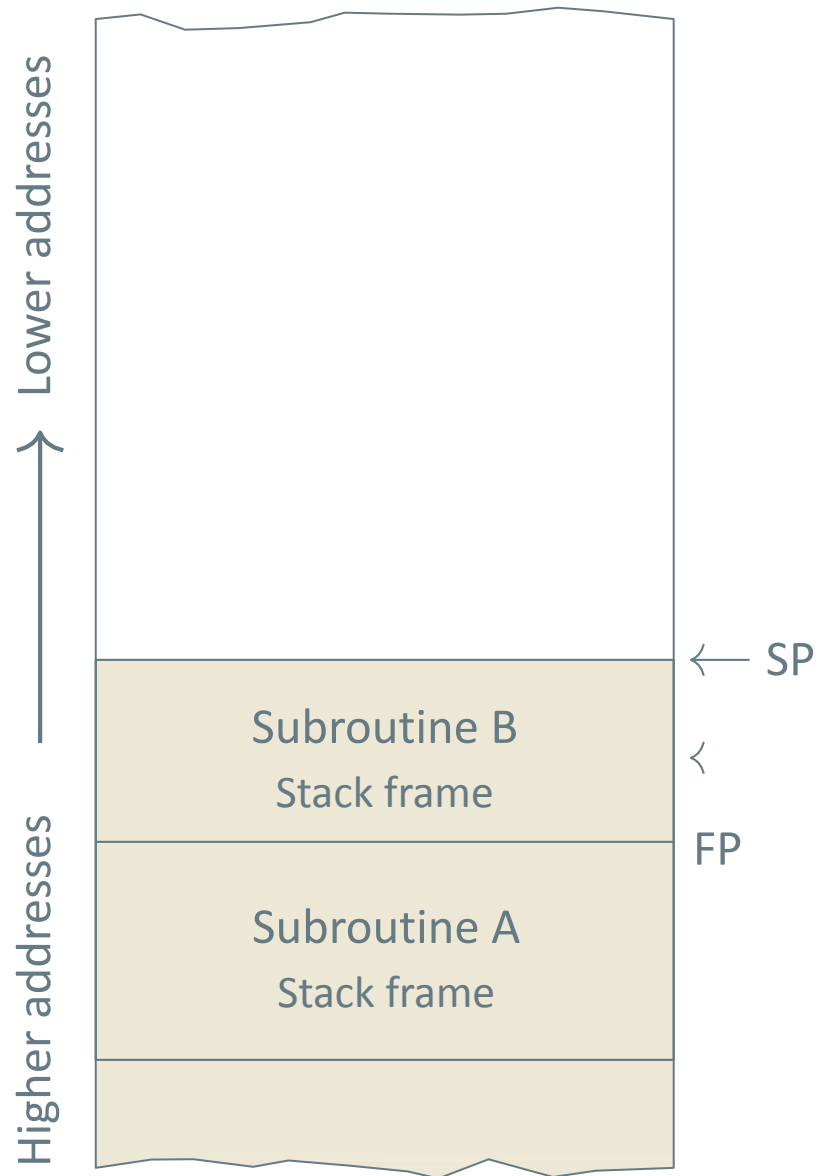


STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

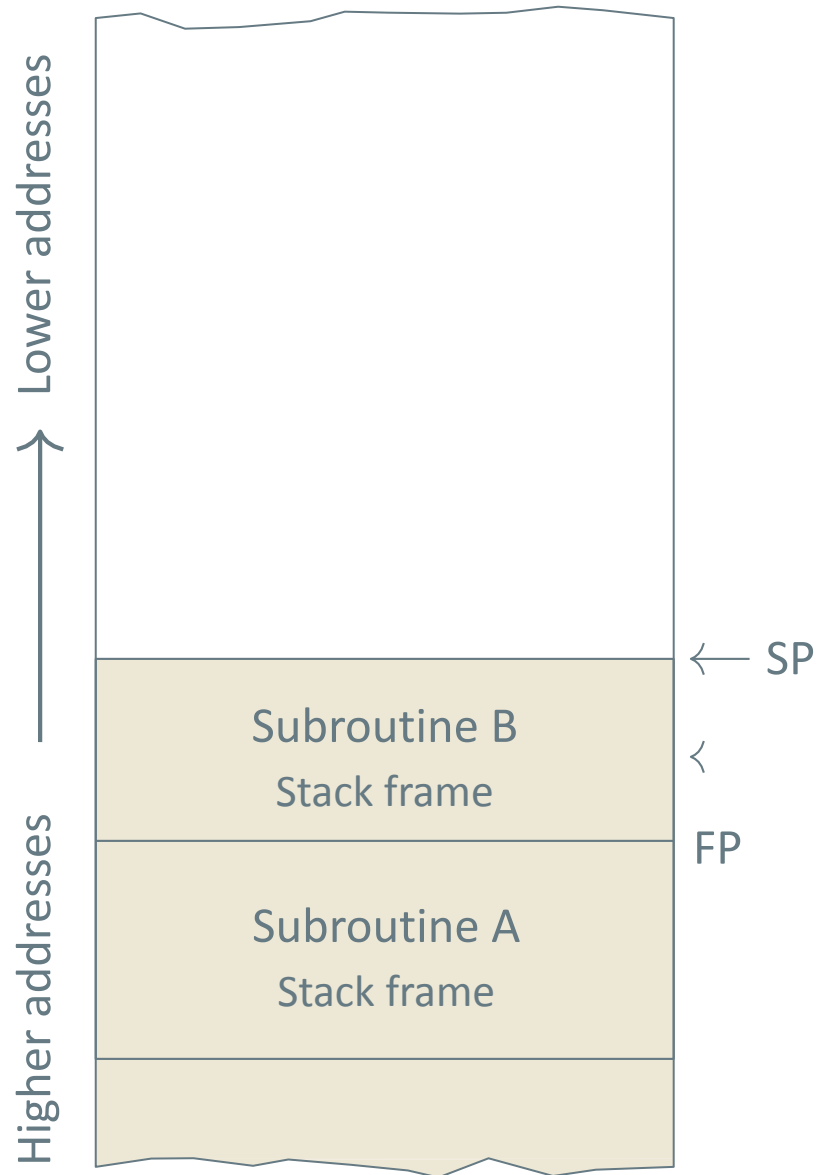
STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

Caller:

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

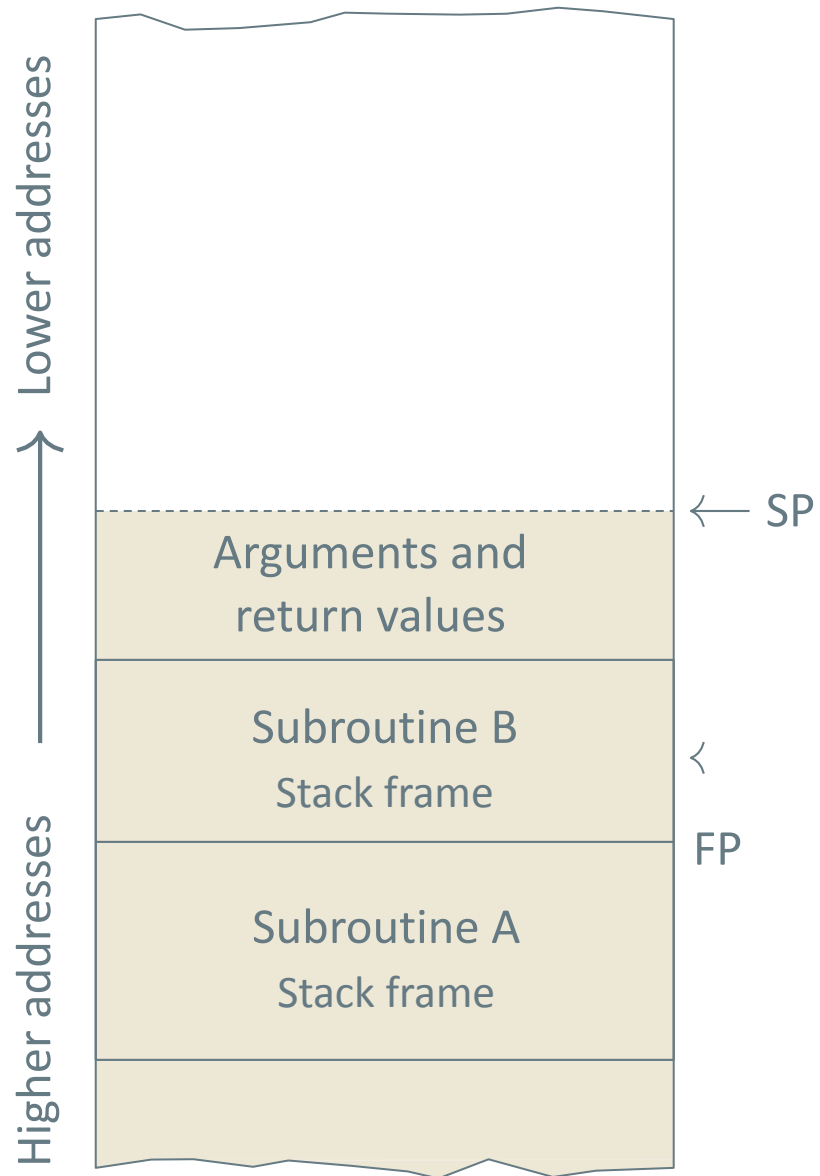


Making the call:

Caller:

- Push arguments on the stack

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

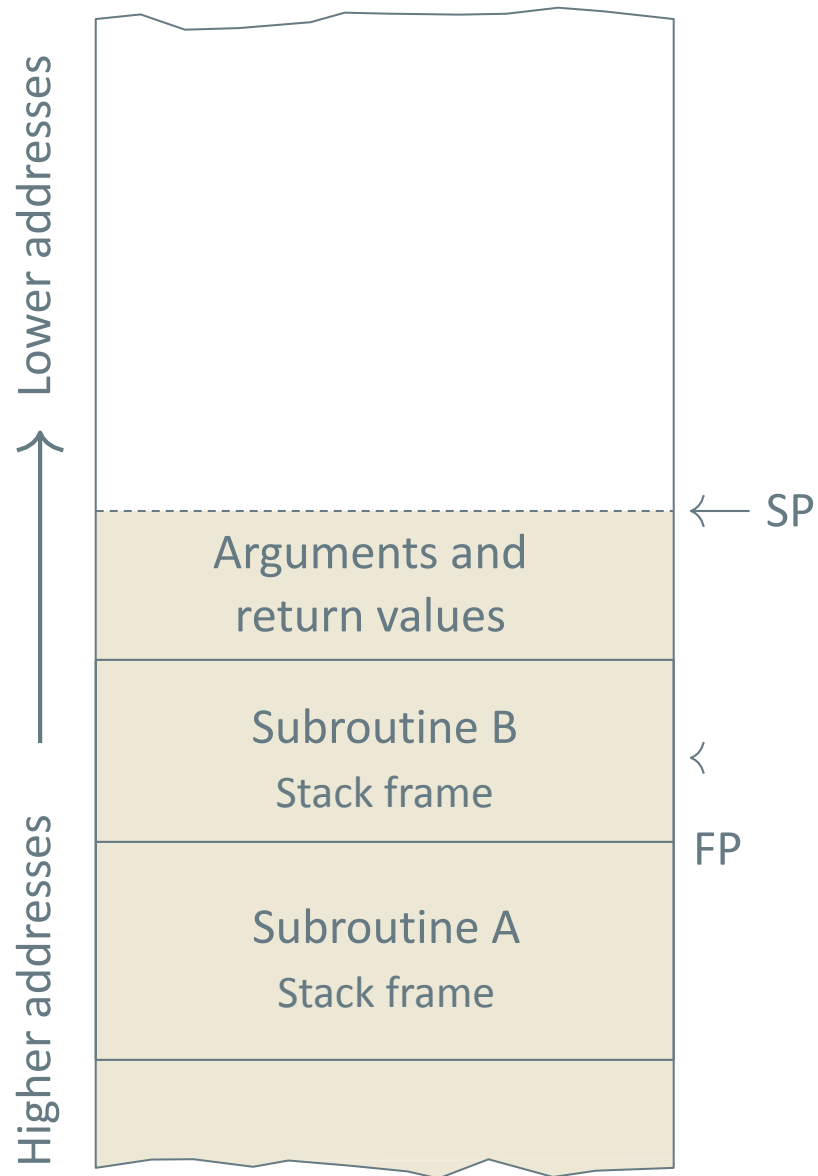


Making the call:

Caller:

- Push arguments on the stack

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

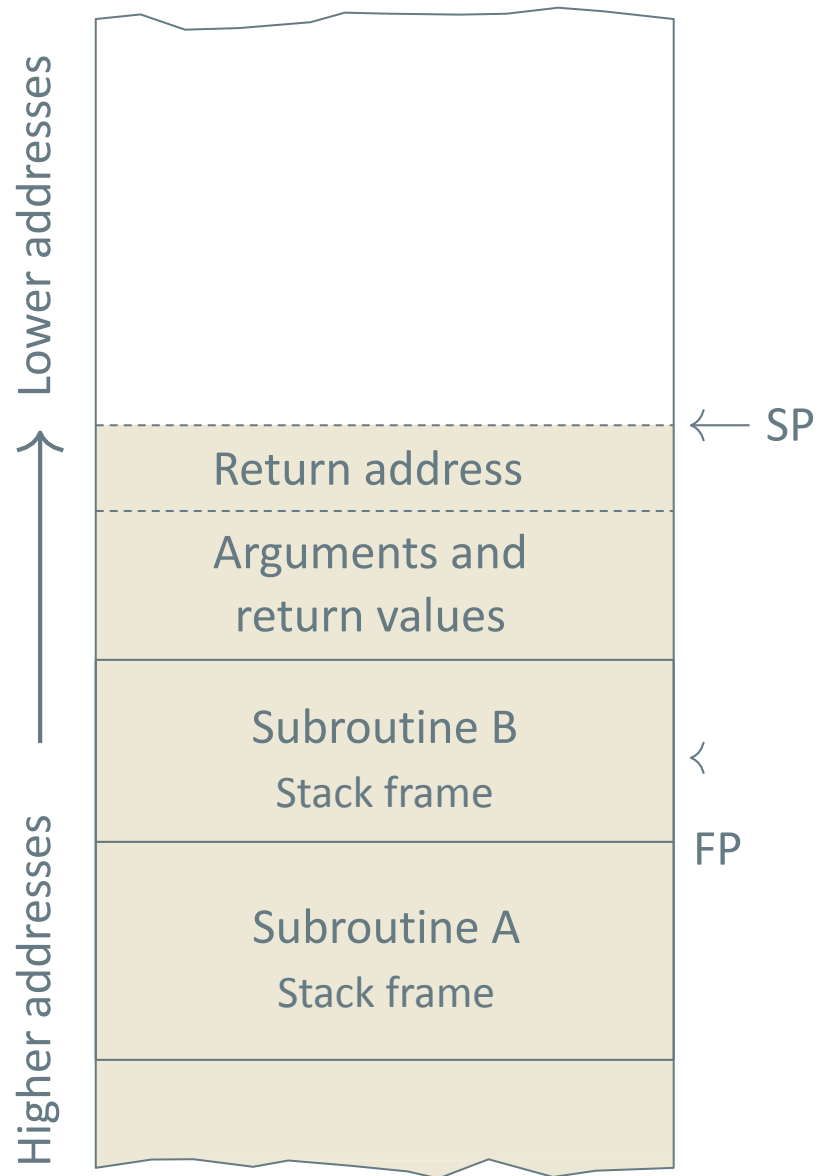


Making the call:

Caller:

- Push arguments on the stack
- Make subroutine call

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

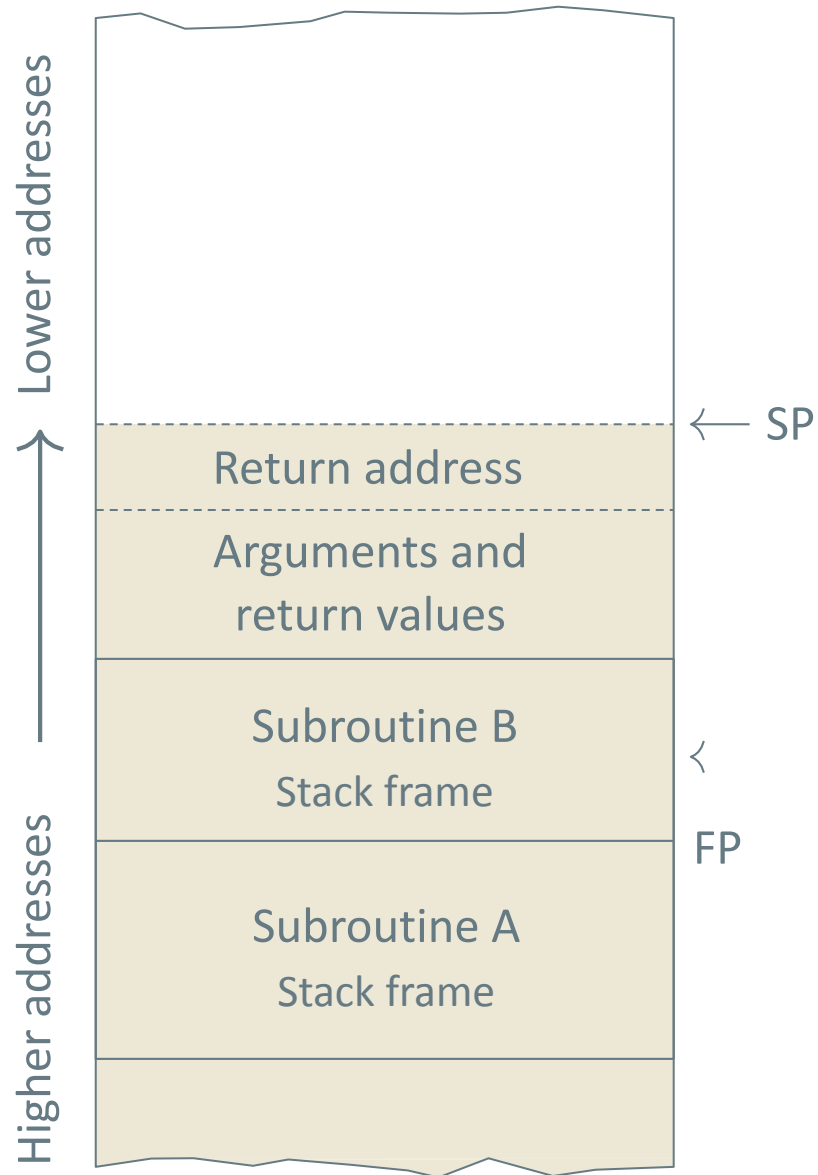


Making the call:

Caller:

- Push arguments on the stack
- Make subroutine call

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



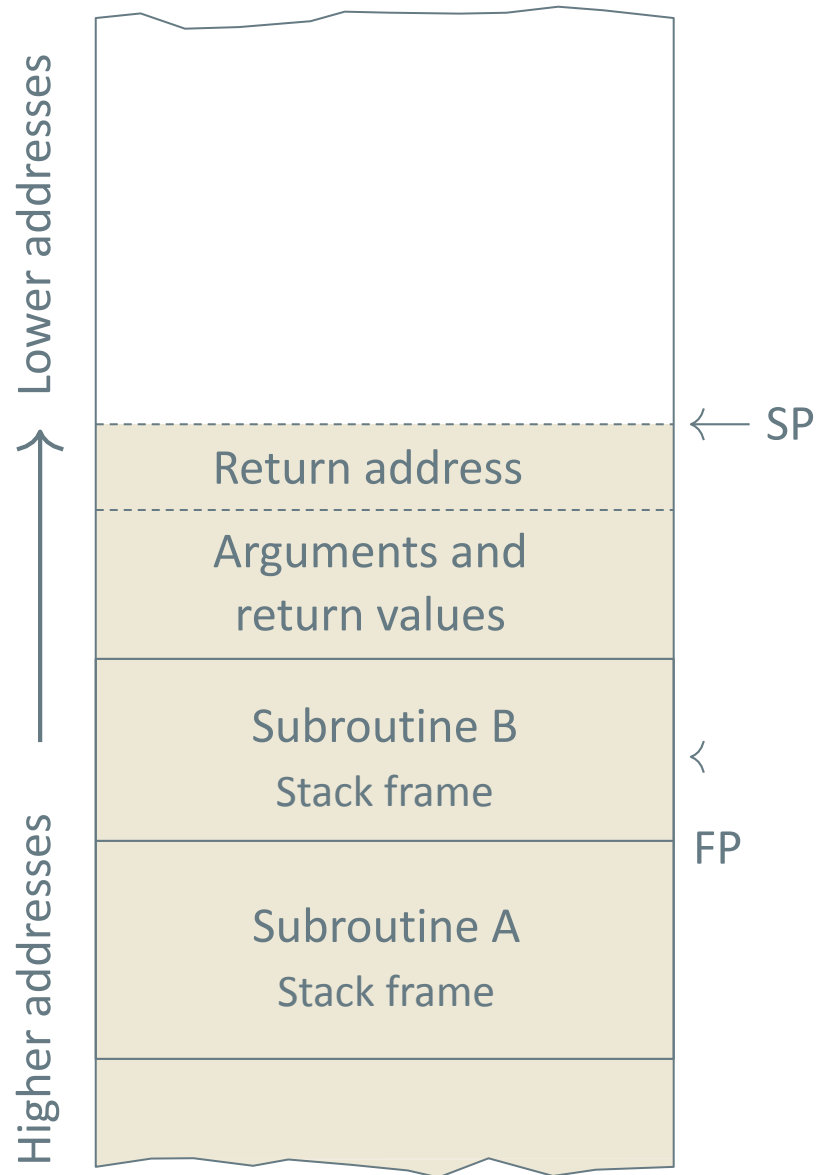
Making the call:

Caller:

- Push arguments on the stack
- Make subroutine call

Callee:

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

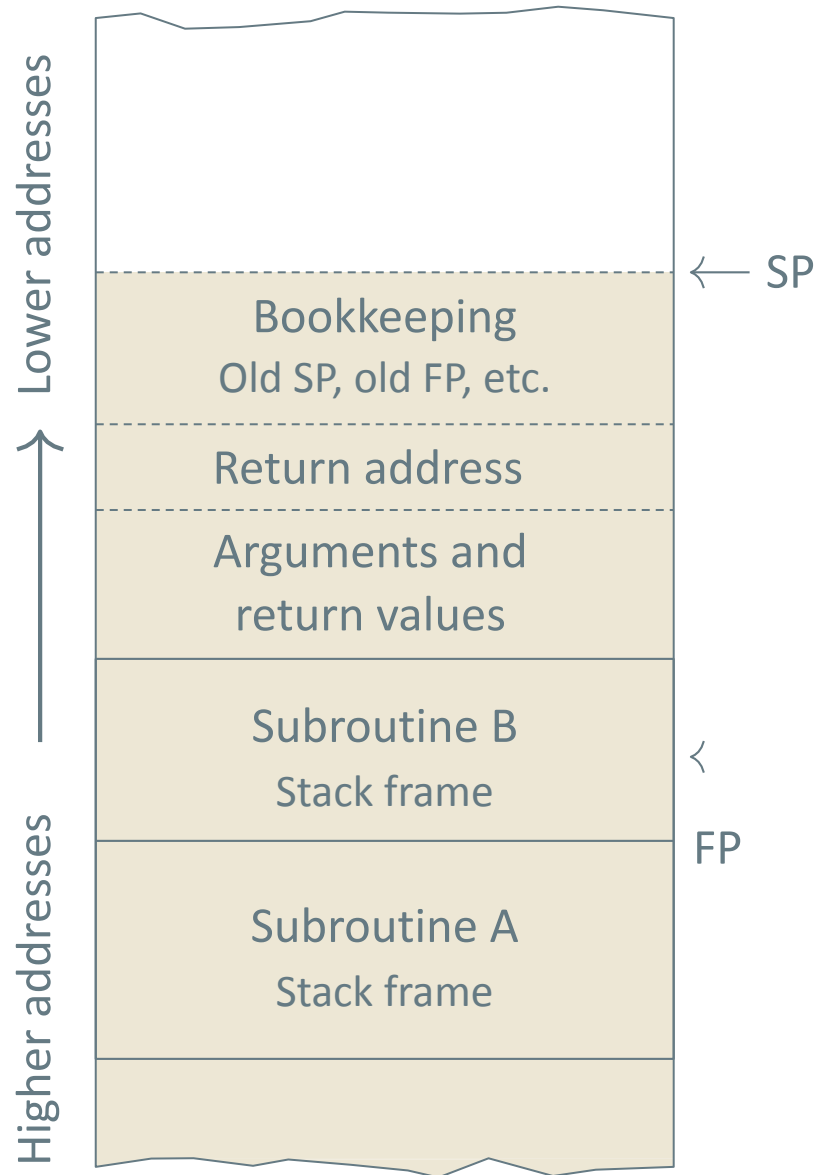
Caller:

- Push arguments on the stack
- Make subroutine call

Callee:

- Push stack pointer and frame pointer on stack

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

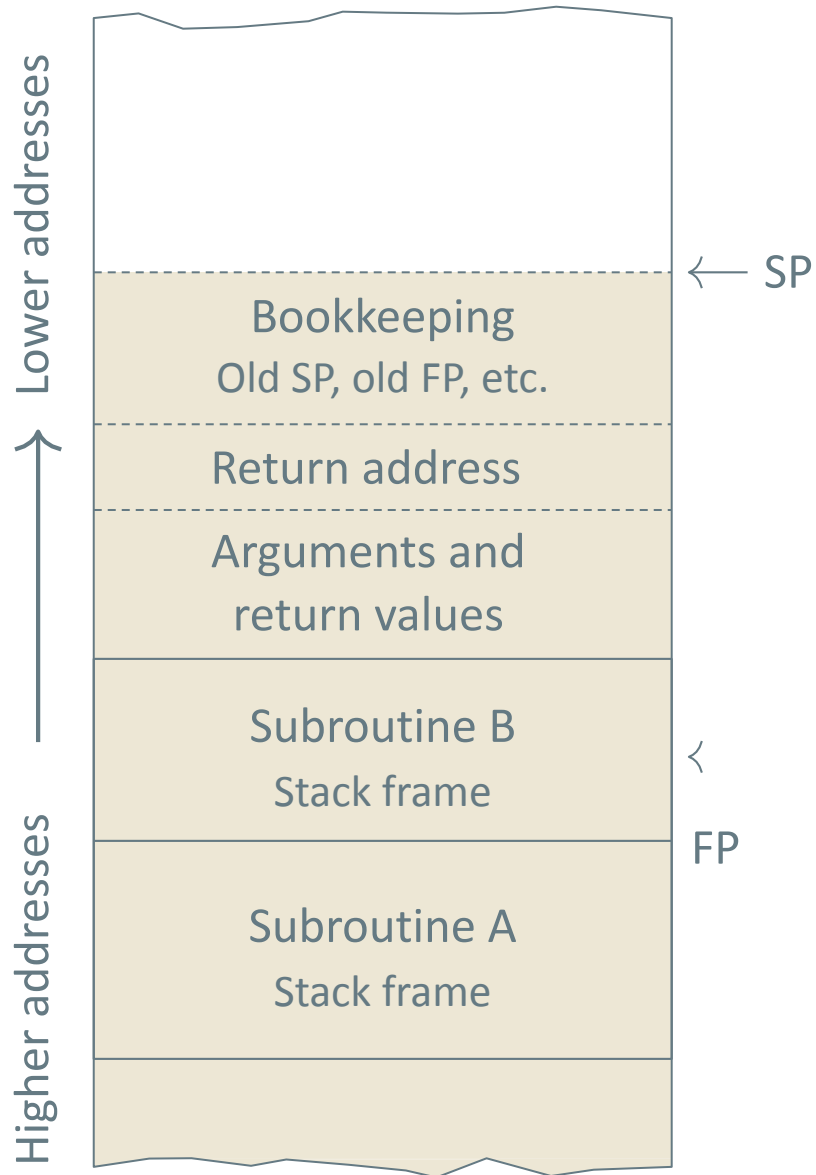
Caller:

- Push arguments on the stack
- Make subroutine call

Callee:

- Push stack pointer and frame pointer on stack

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

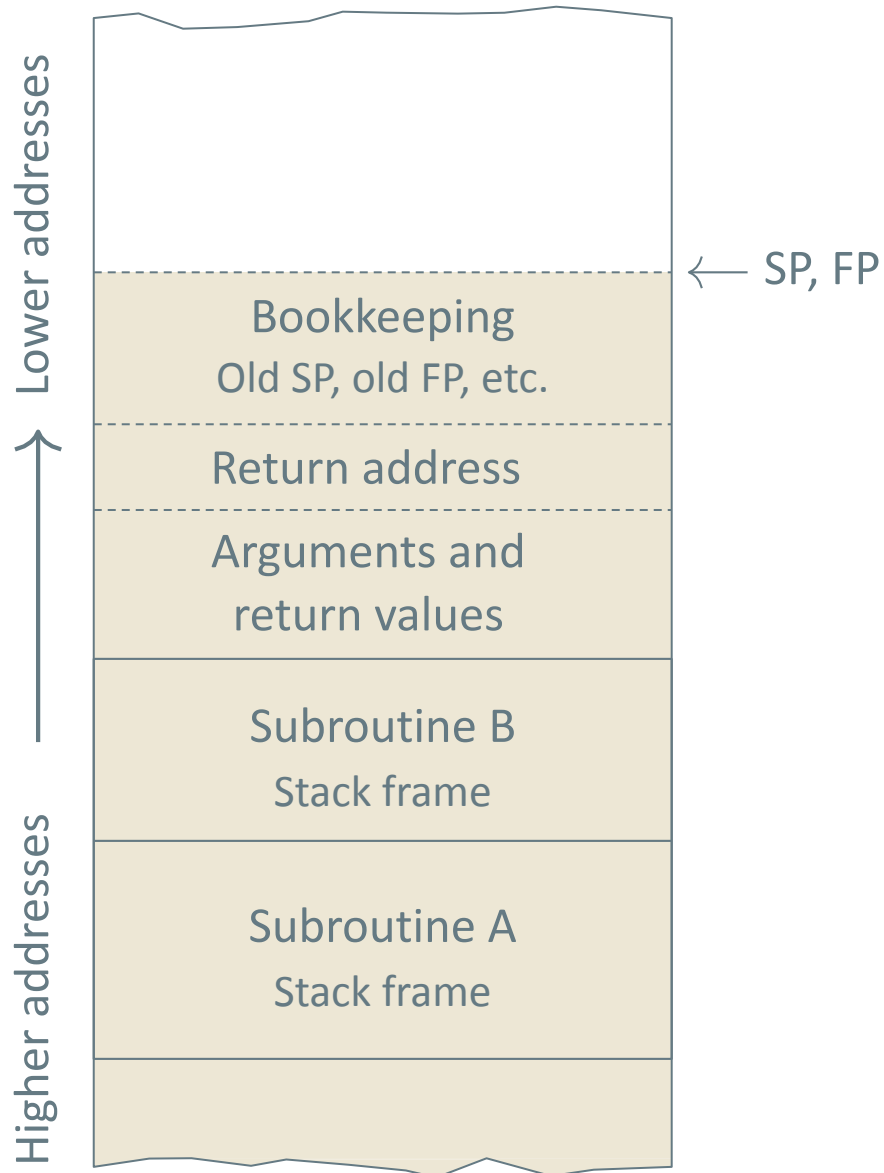
Caller:

- Push arguments on the stack
- Make subroutine call

Callee:

- Push stack pointer and frame pointer on stack
- Update frame pointer

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

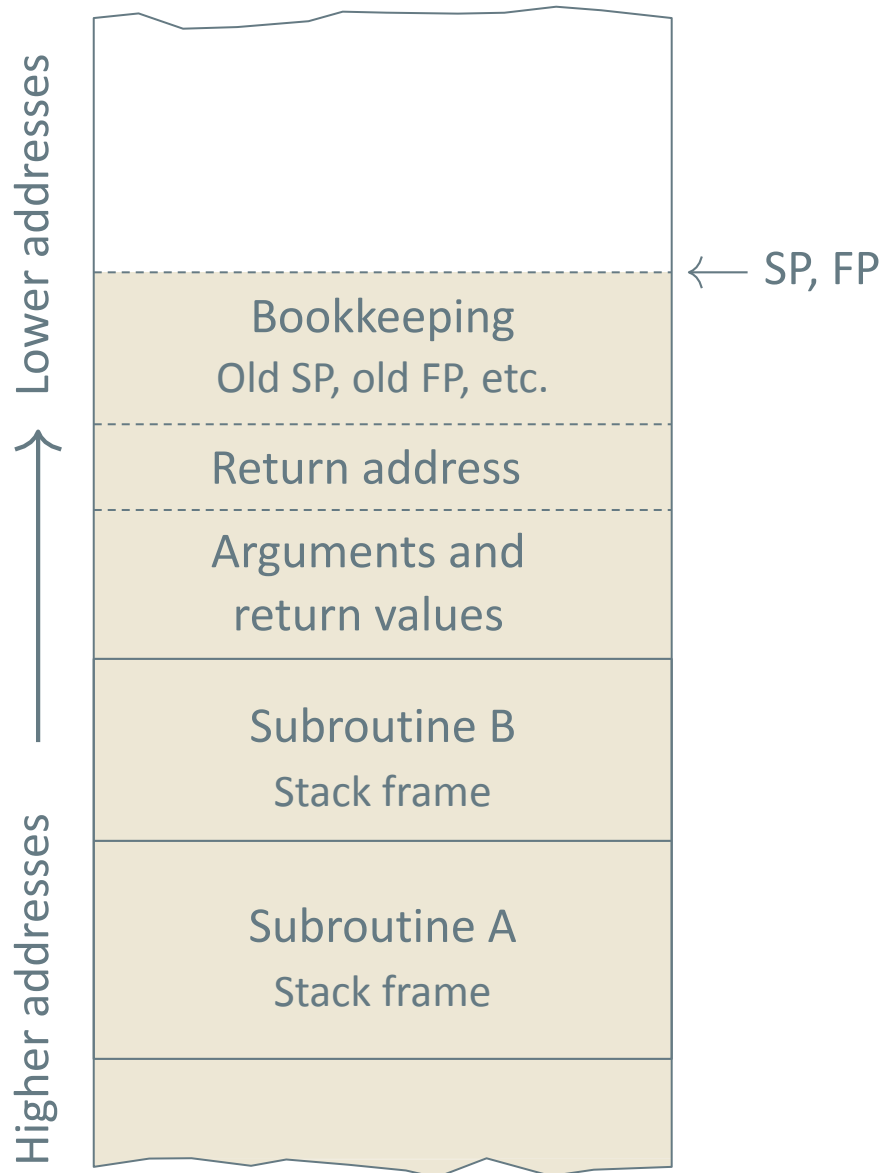
Caller:

- Push arguments on the stack
- Make subroutine call

Callee:

- Push stack pointer and frame pointer on stack
- Update frame pointer

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

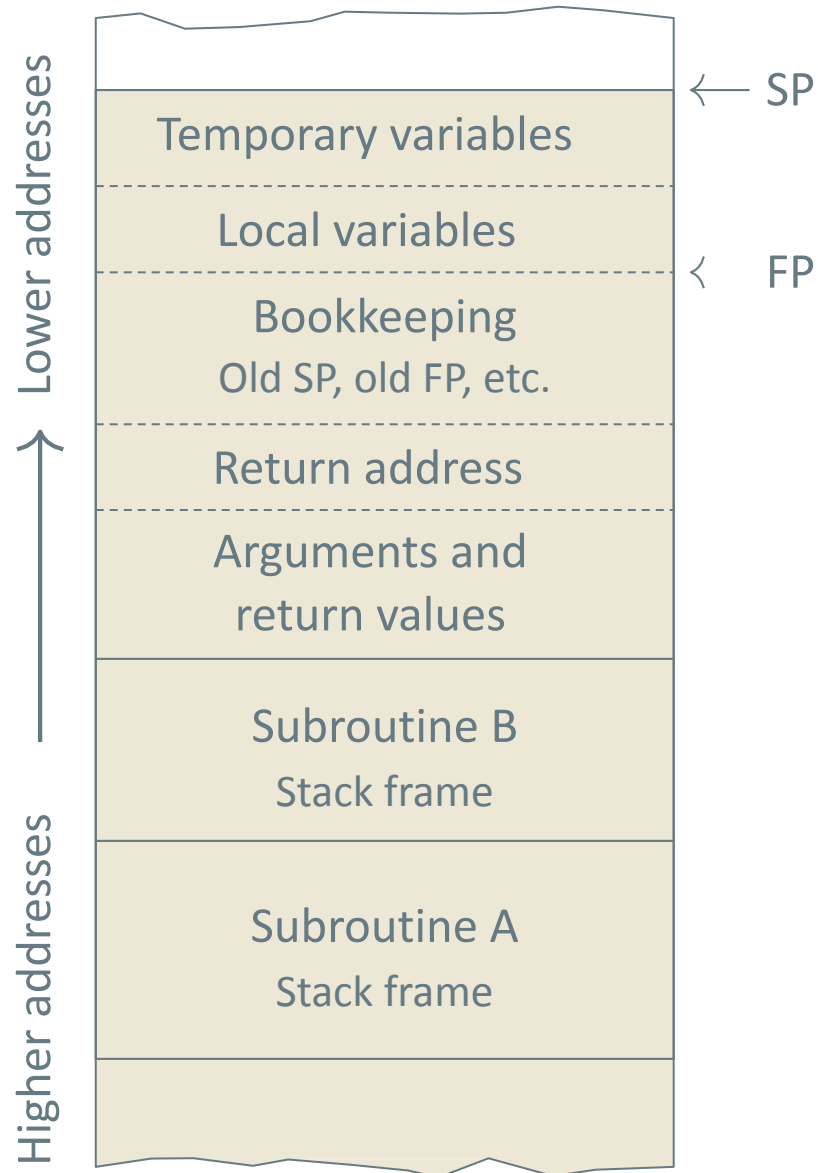
Caller:

- Push arguments on the stack
- Make subroutine call

Callee:

- Push stack pointer and frame pointer on stack
- Update frame pointer
- Allocate local variables and temporary working space, update stack pointer

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Making the call:

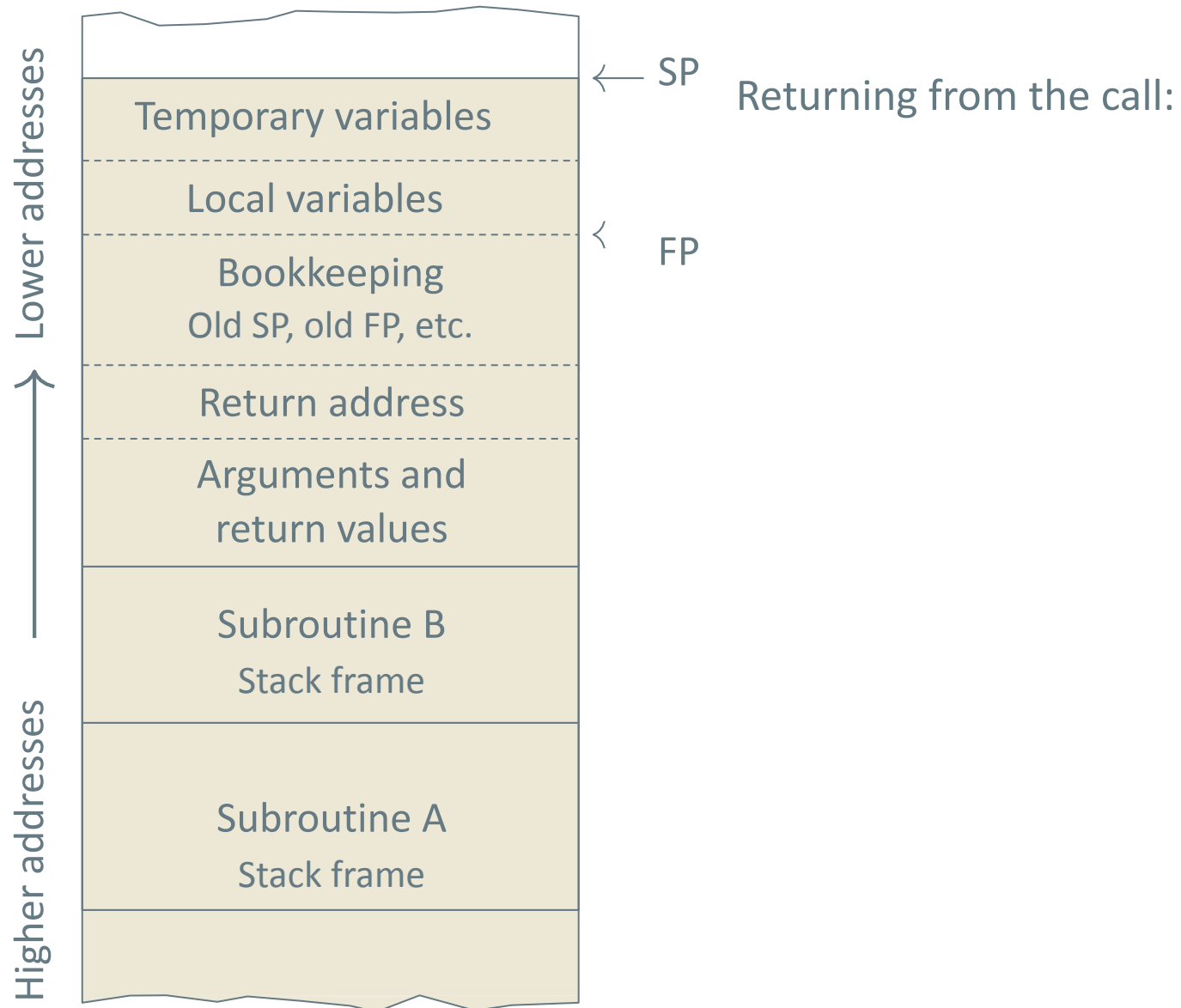
Caller:

- Push arguments on the stack
- Make subroutine call

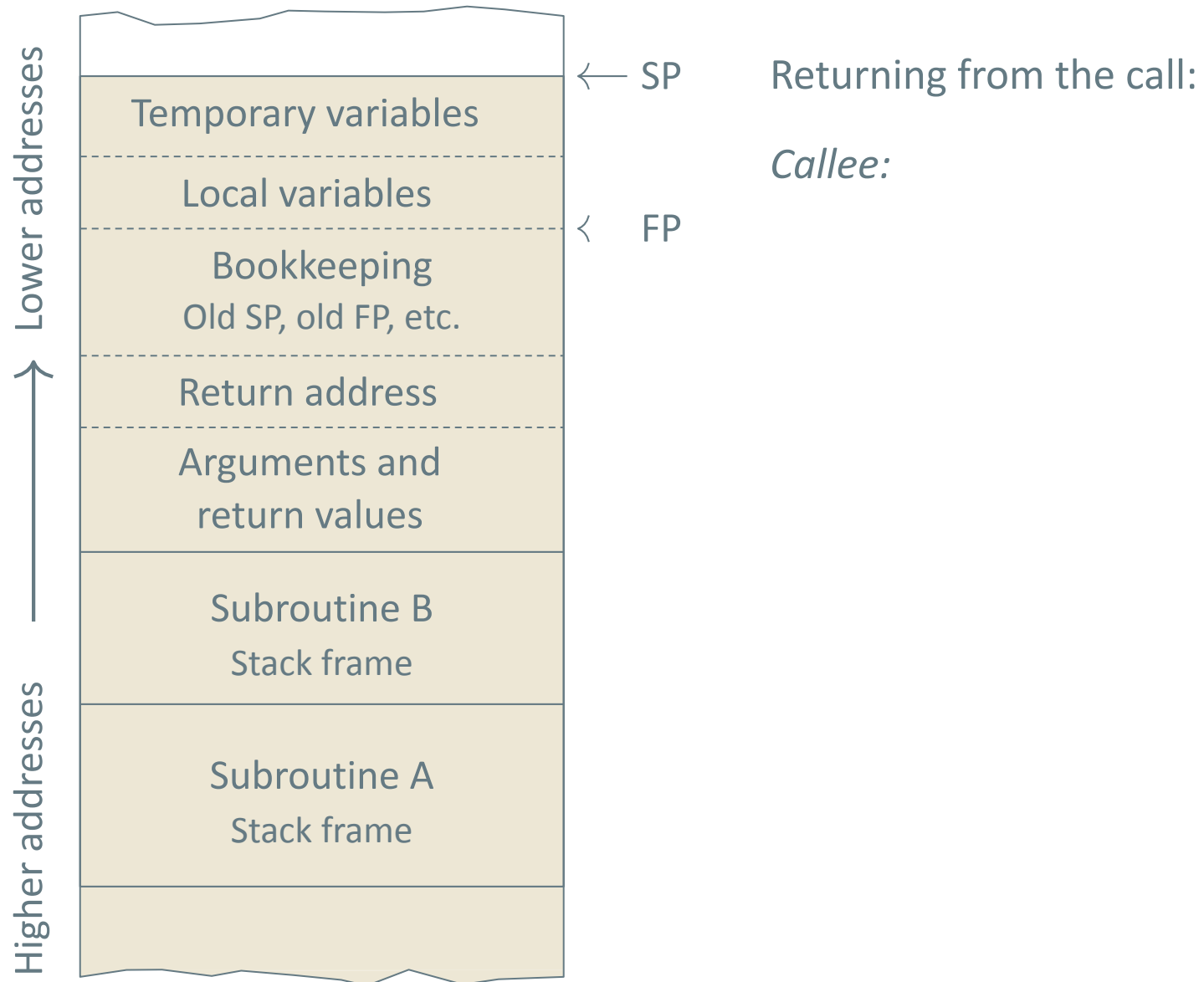
Callee:

- Push stack pointer and frame pointer on stack
- Update frame pointer
- Allocate local variables and temporary working space, update stack pointer

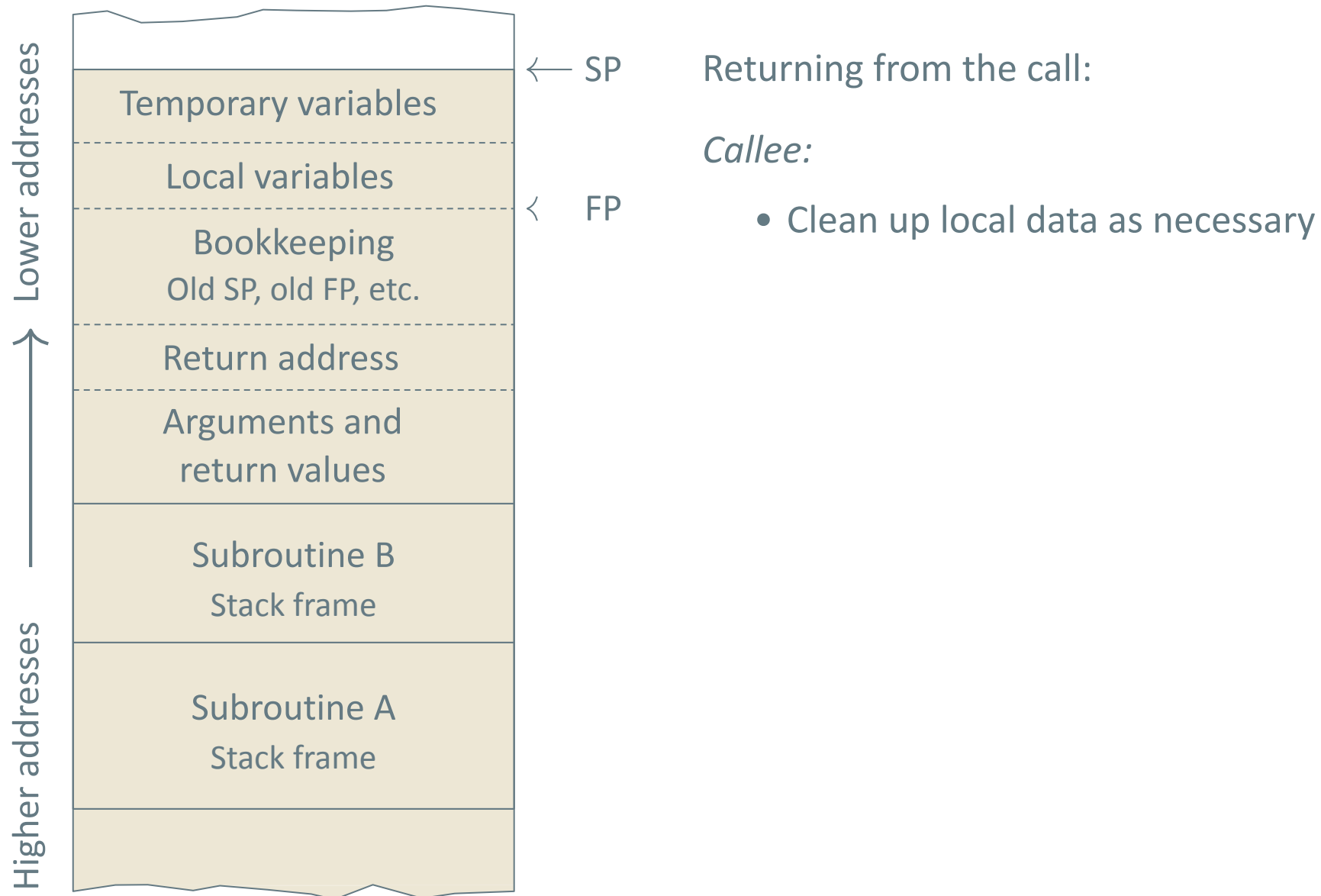
STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



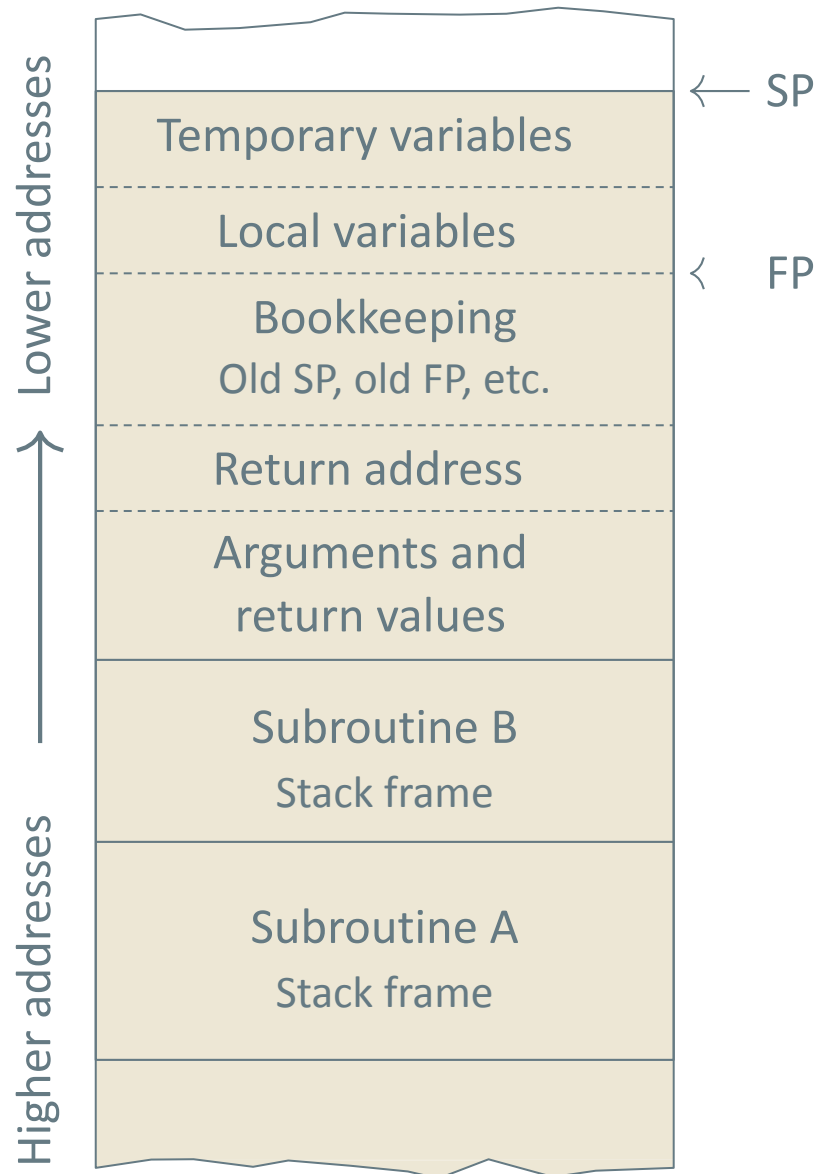
STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

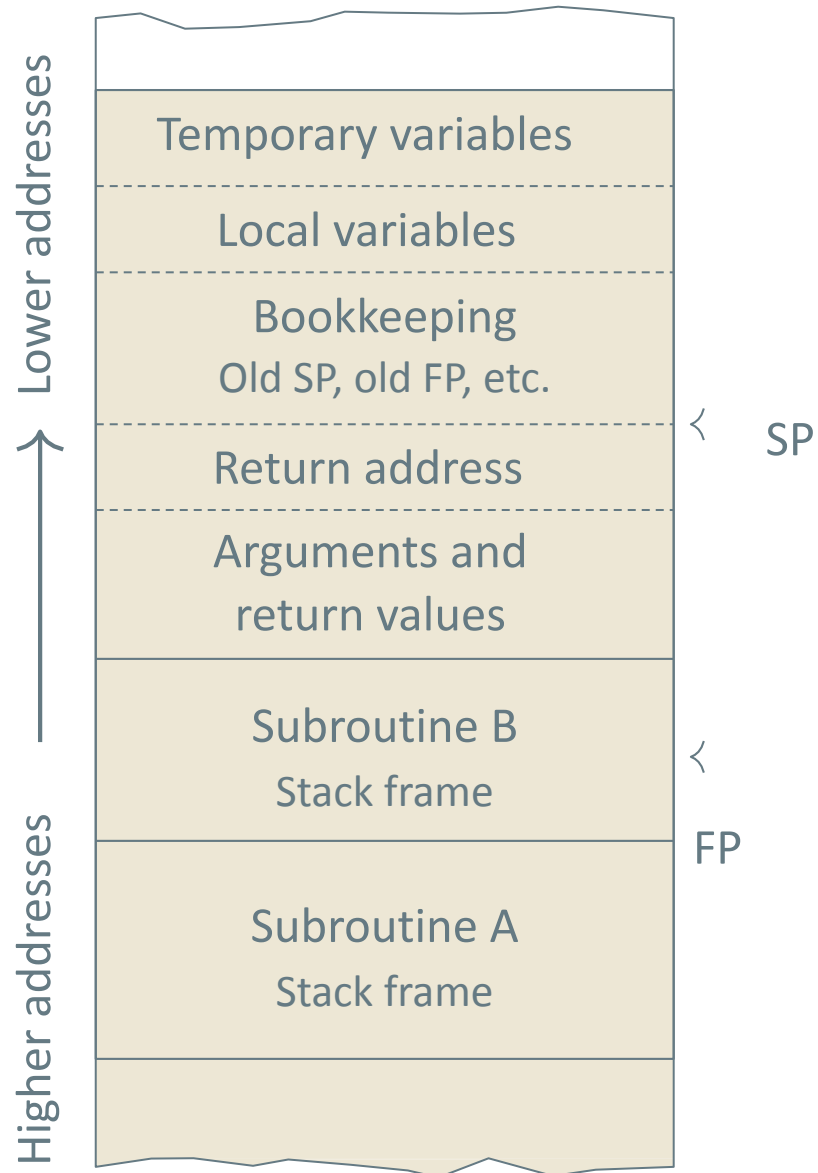


Returning from the call:

Callee:

- Clean up local data as necessary
- Restore stack pointer and frame pointer

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

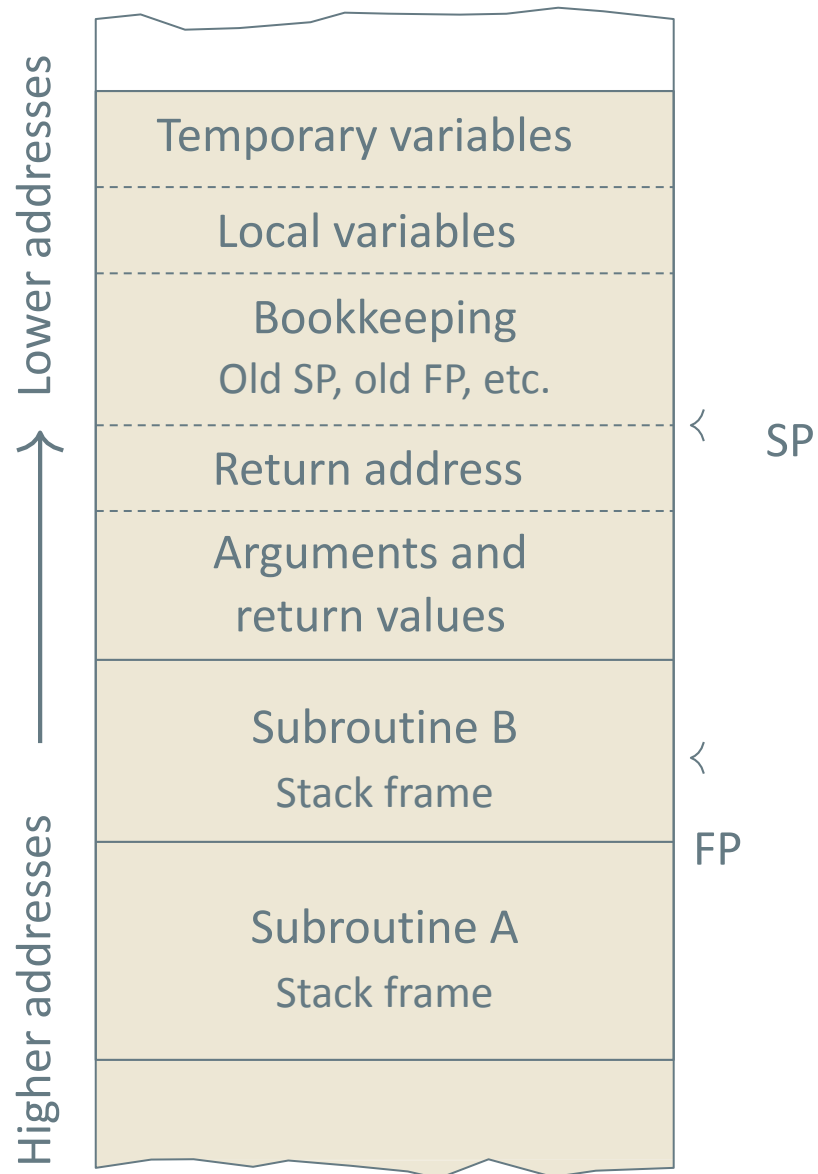


Returning from the call:

Callee:

- Clean up local data as necessary
- Restore stack pointer and frame pointer

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

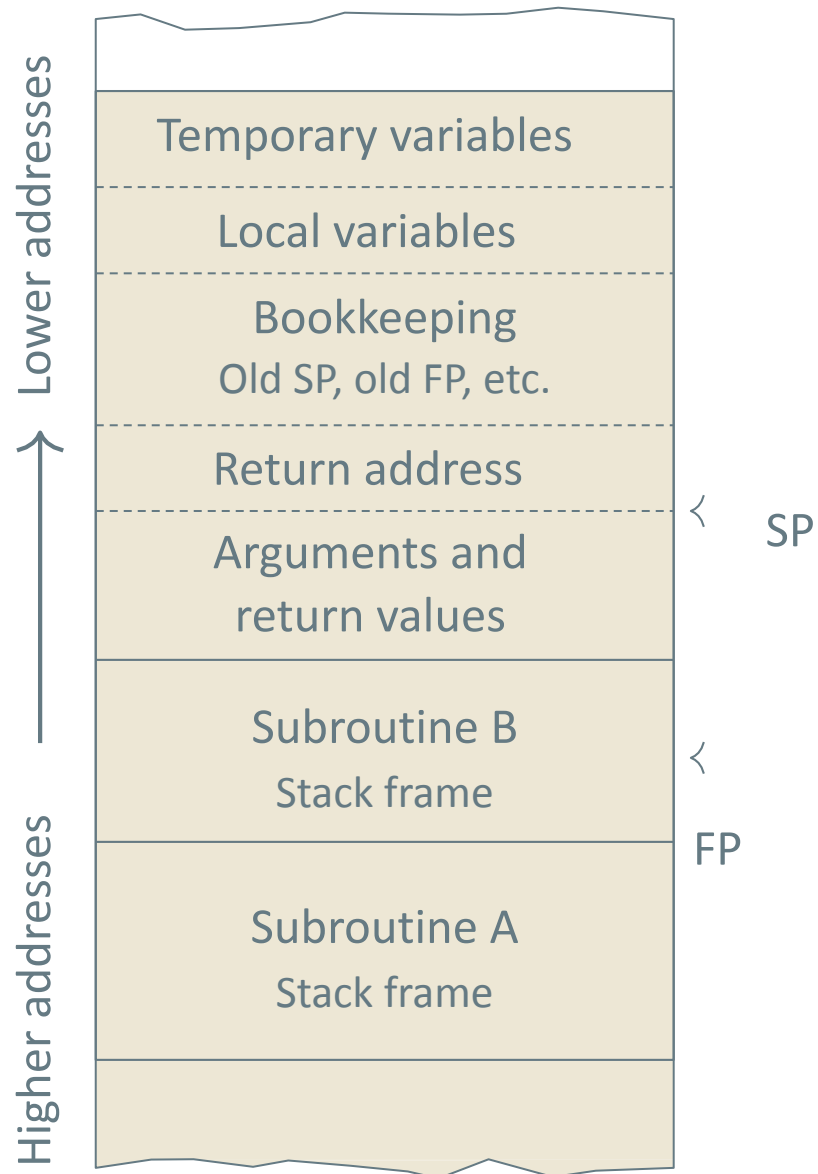


Returning from the call:

Callee:

- Clean up local data as necessary
- Restore stack pointer and frame pointer
- Return

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL

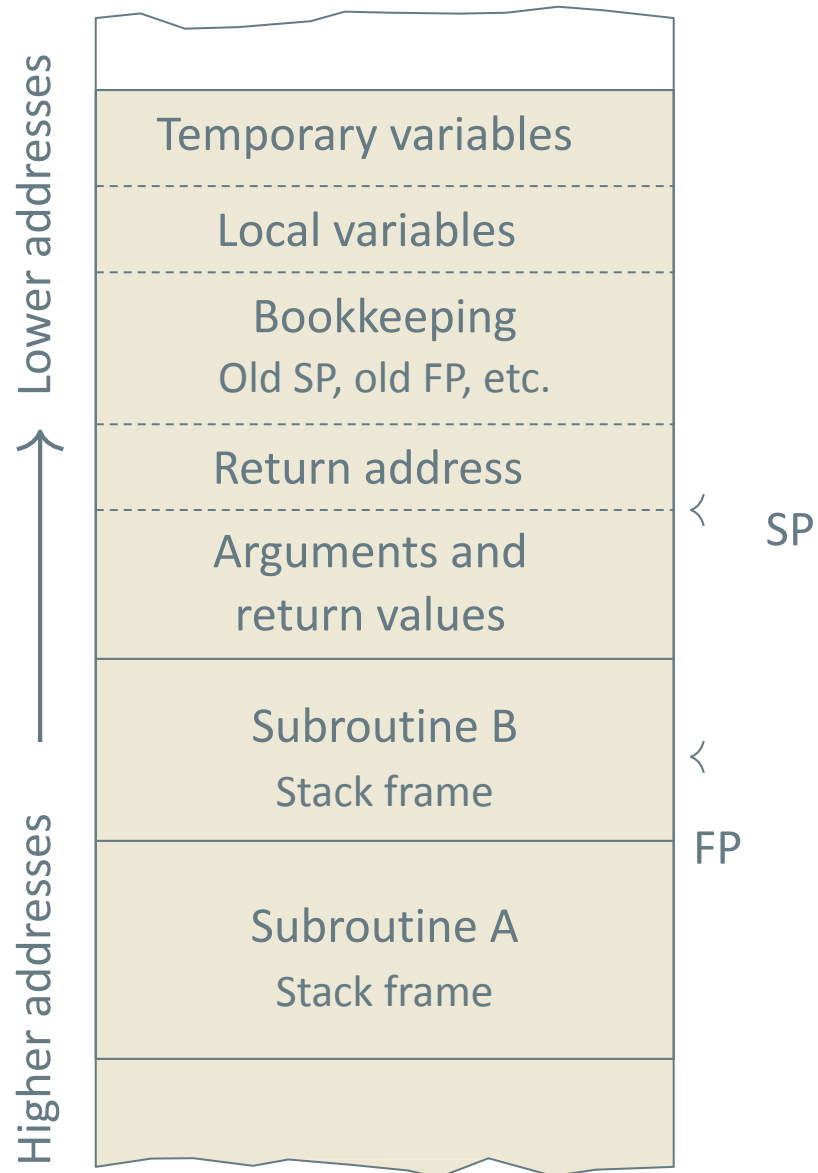


Returning from the call:

Callee:

- Clean up local data as necessary
- Restore stack pointer and frame pointer
- Return

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Returning from the call:

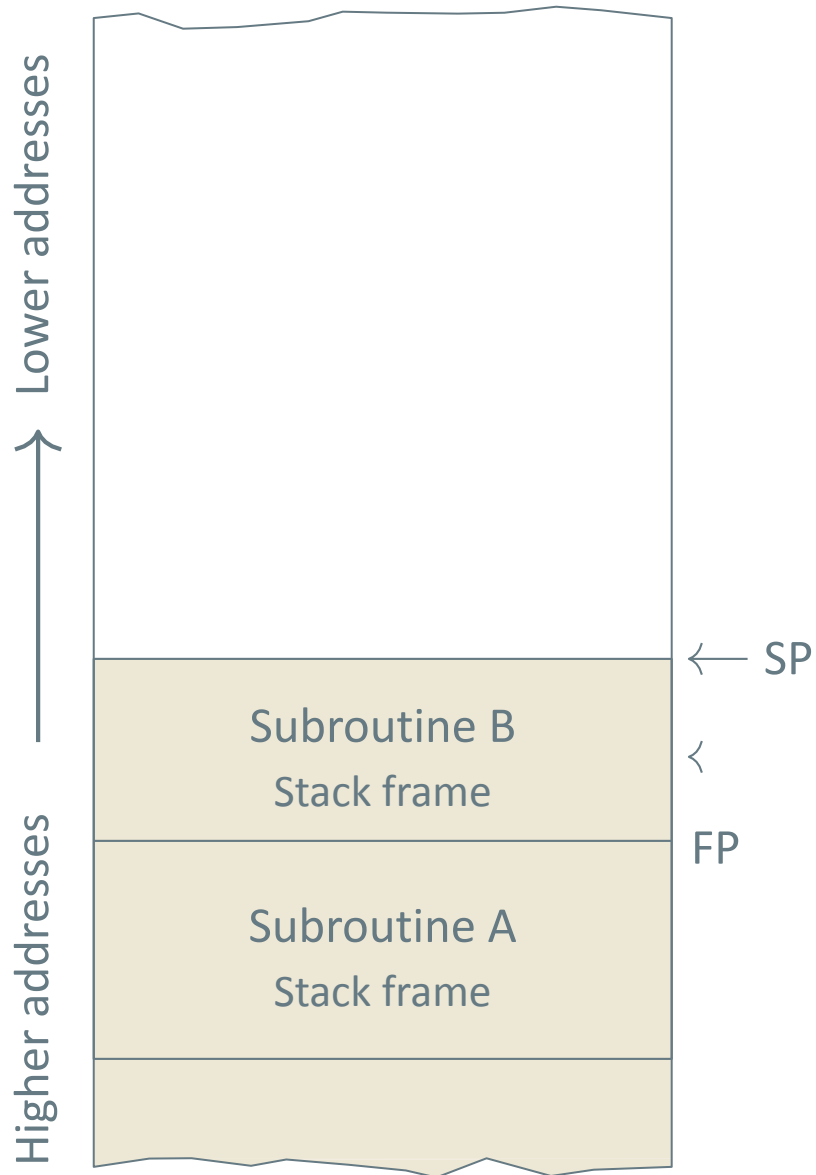
Callee:

- Clean up local data as necessary
- Restore stack pointer and frame pointer
- Return

Caller:

- Retrieve return value from the stack and restore stack pointer to top of previous stack frame

STACK FRAME BEFORE, DURING & AFTER SUBROUTINE CALL



Returning from the call:

Callee:

- Clean up local data as necessary
- Restore stack pointer and frame pointer
- Return

Caller:

- Retrieve return value from the stack and restore stack pointer to top of previous stack frame

Name, binding and scope

- **HEAP ALLOCATION**

- A heap is a region of storage in which blocks can be allocated and deallocated at arbitrary times. Heaps are required for the dynamically allocated pieces of linked data structures, and for objects like fully general character strings, lists, and sets, whose size may change as a result of an assignment statement or other update operation.



Figure 3.2 Fragmentation. The shaded blocks are in use; the clear blocks are free. Cross-hatched space at the ends of in-use blocks represents internal fragmentation. The discontinuous free blocks indicate external fragmentation. While there is more than enough total free space remaining to satisfy an allocation request of the illustrated size, no single remaining block is large enough.

Name, binding and scope

- HEAP ALLOCATION

- There are many possible strategies to manage space in a heap, but with space two issues were found:
- Internal fragmentation
 - when a storage-management algorithm allocates a block that is larger than required to hold a given object; the extra space is then unused.
- External fragmentation
 - when the blocks that have been assigned to active objects are scattered through the heap in such a way that the remaining, unused space is composed of multiple blocks: there may be quite a lot of free space, but no one piece of it may be large enough to satisfy some future request.

Name, binding and scope

- **HEAP ALLOCATION**

- Heap management

- Free list: List of blocks of free memory
 - The allocation algorithm searches for a block of adequate size to accommodate the allocation request.



FIRST-FIT AND BEST-FIT ALLOCATION

General allocation strategy

- Find a free block that is at least as big as the requested amount of memory.
- Mark requested number of bytes (plus padding) as allocated.
- Return rest of the free block to free list.

FIRST-FIT AND BEST-FIT ALLOCATION

General allocation strategy

- Find a free block that is at least as big as the requested amount of memory.
- Mark requested number of bytes (plus padding) as allocated.
- Return rest of the free block to free list.

First fit: Find the **first block** large enough to accommodate the allocation request.



FIRST-FIT AND BEST-FIT ALLOCATION

General allocation strategy

- Find a free block that is at least as big as the requested amount of memory.
- Mark requested number of bytes (plus padding) as allocated.
- Return rest of the free block to free list.

First fit: Find the **first block** large enough to accommodate the allocation request.



Best fit: Find the **smallest block** large enough to accommodate the request.



COST OF HEAP ALLOCATION

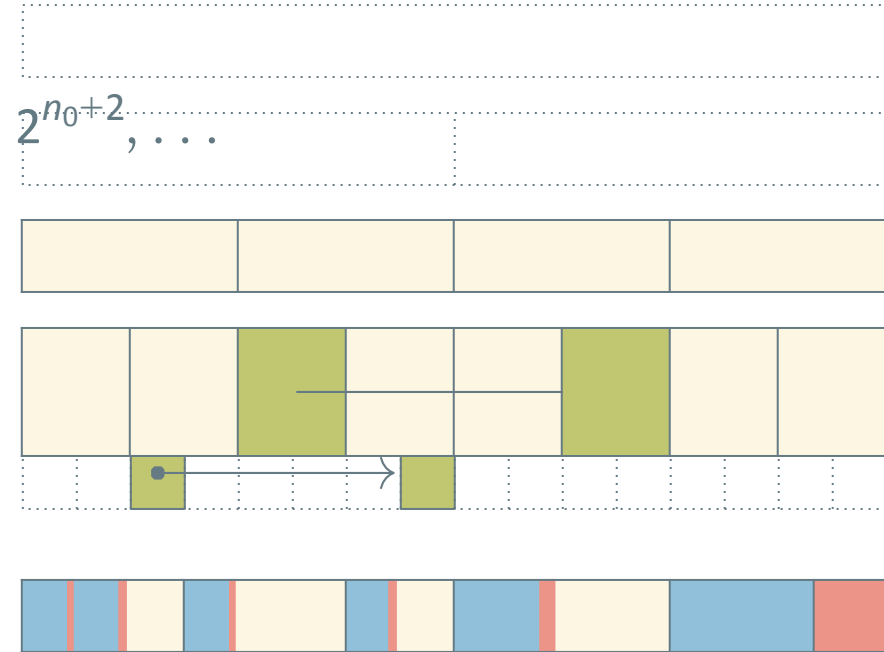
Single free list: Linear cost to find a block to accommodate each request.

COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

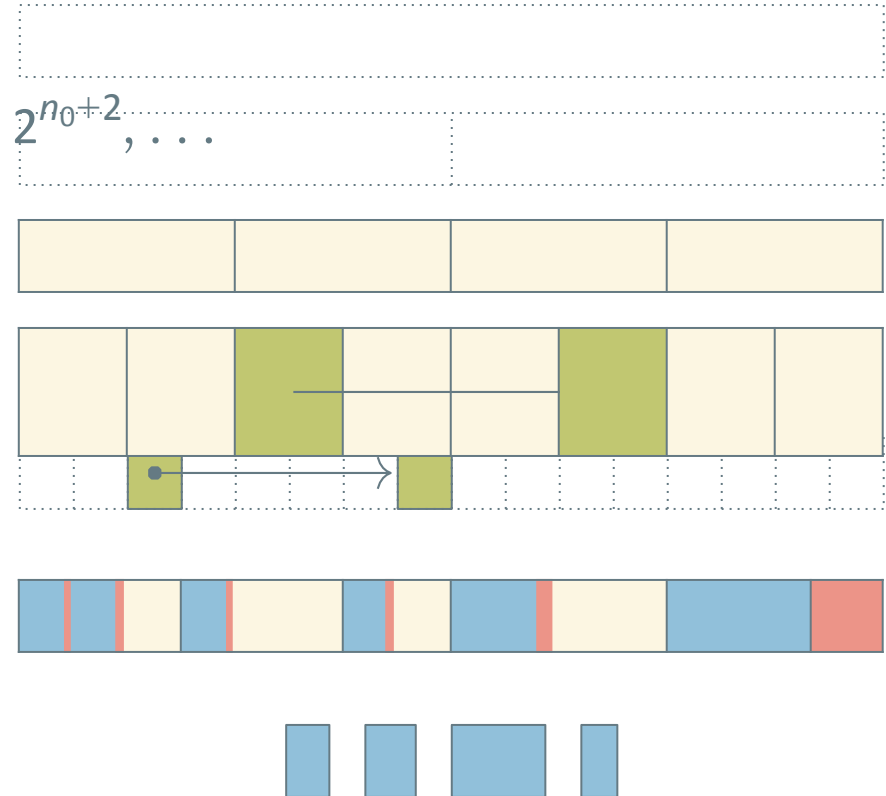


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

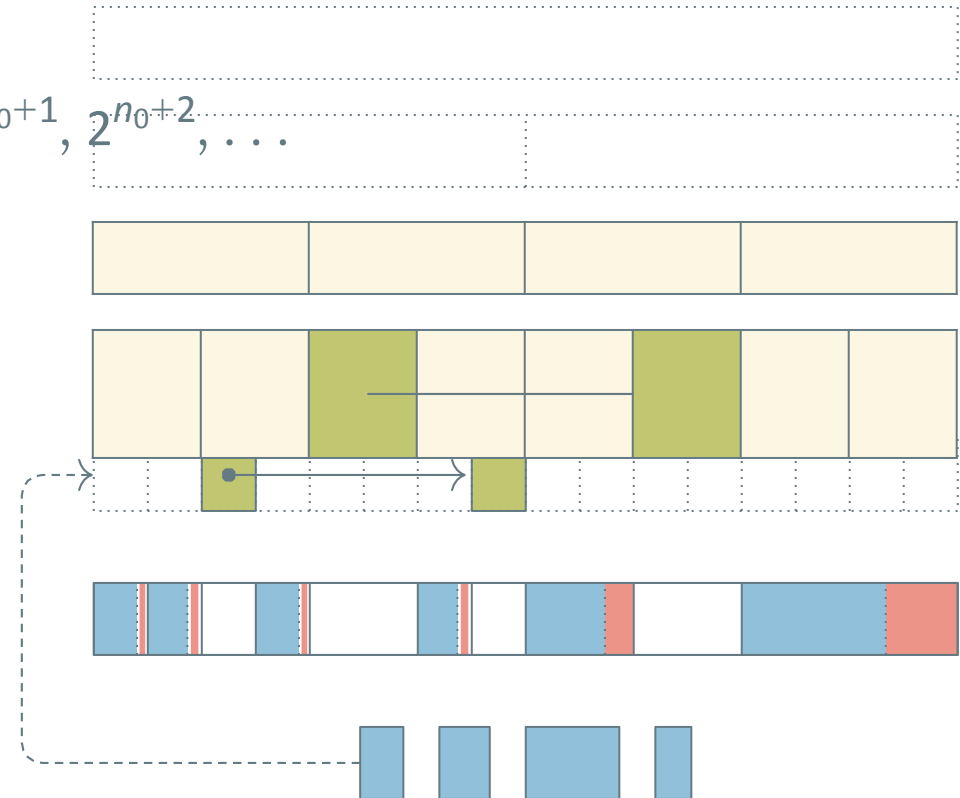


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

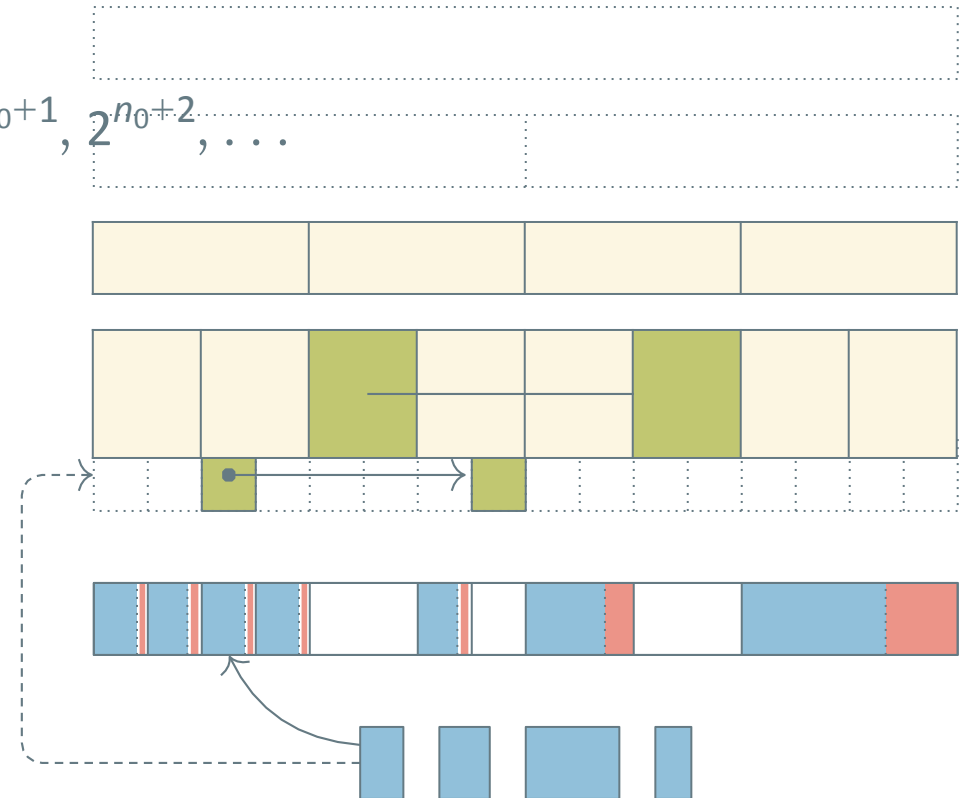


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

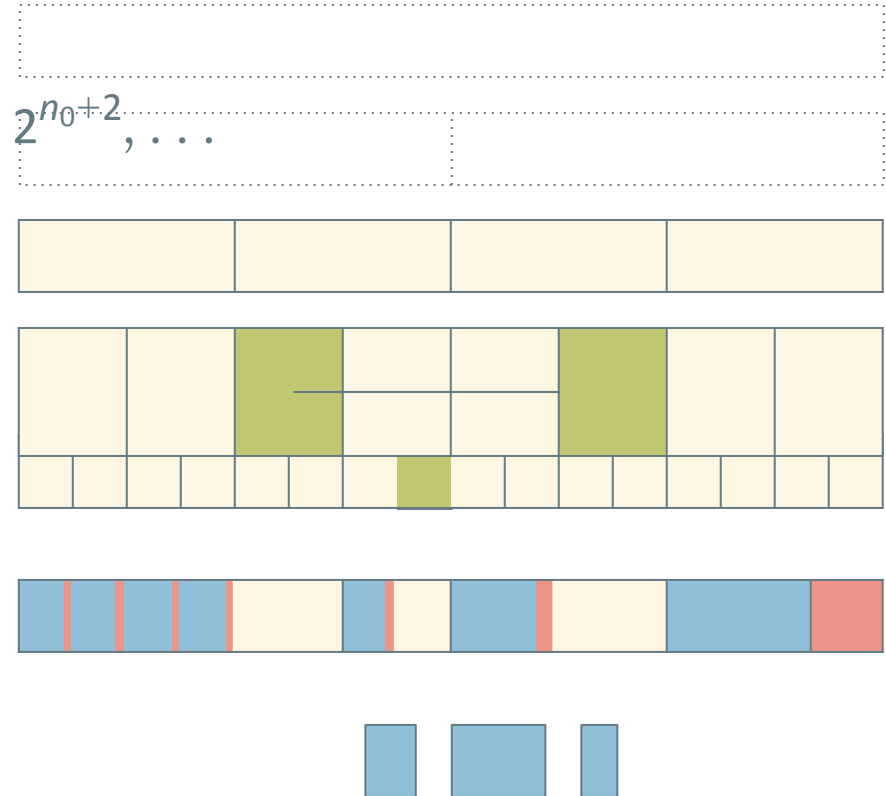


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

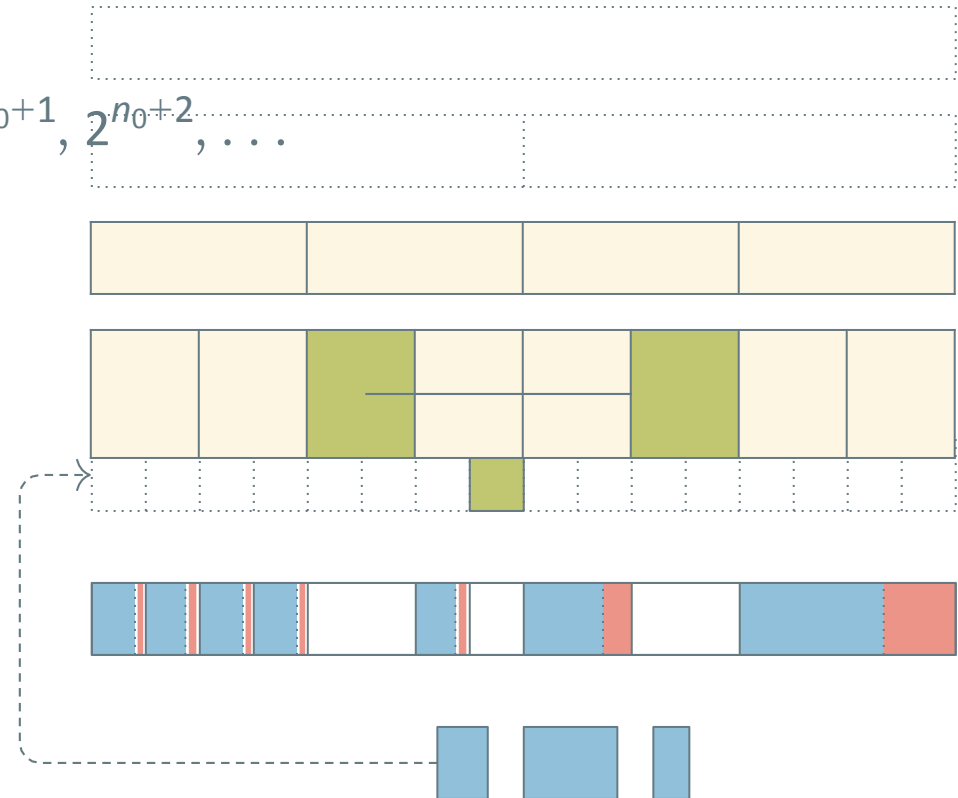


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

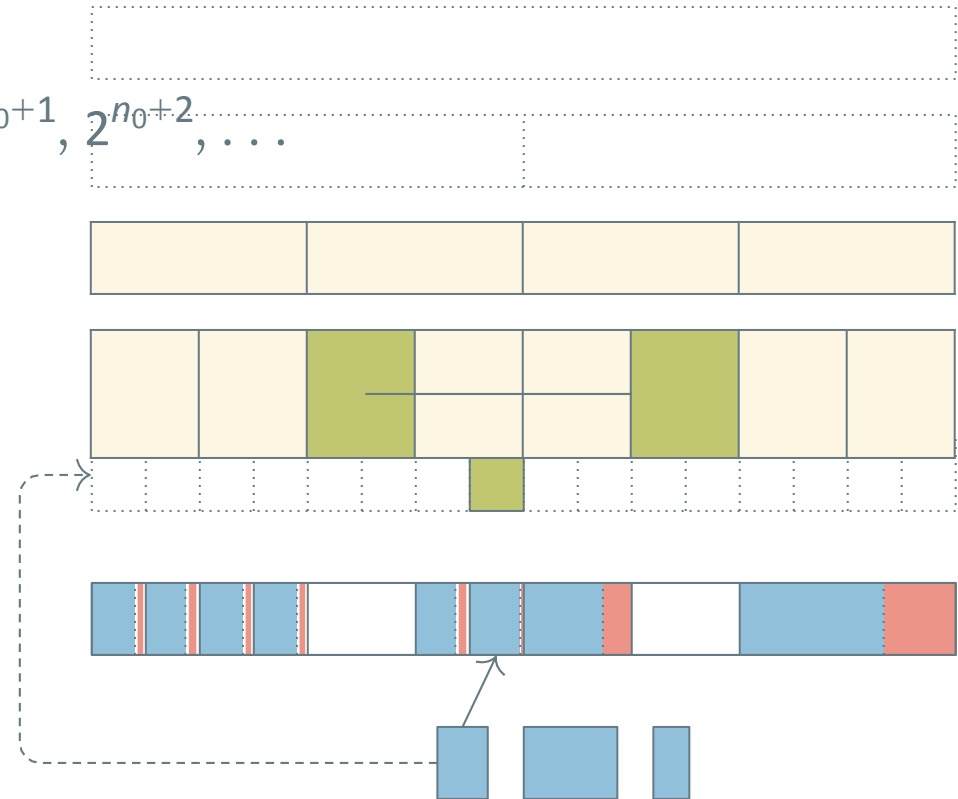


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

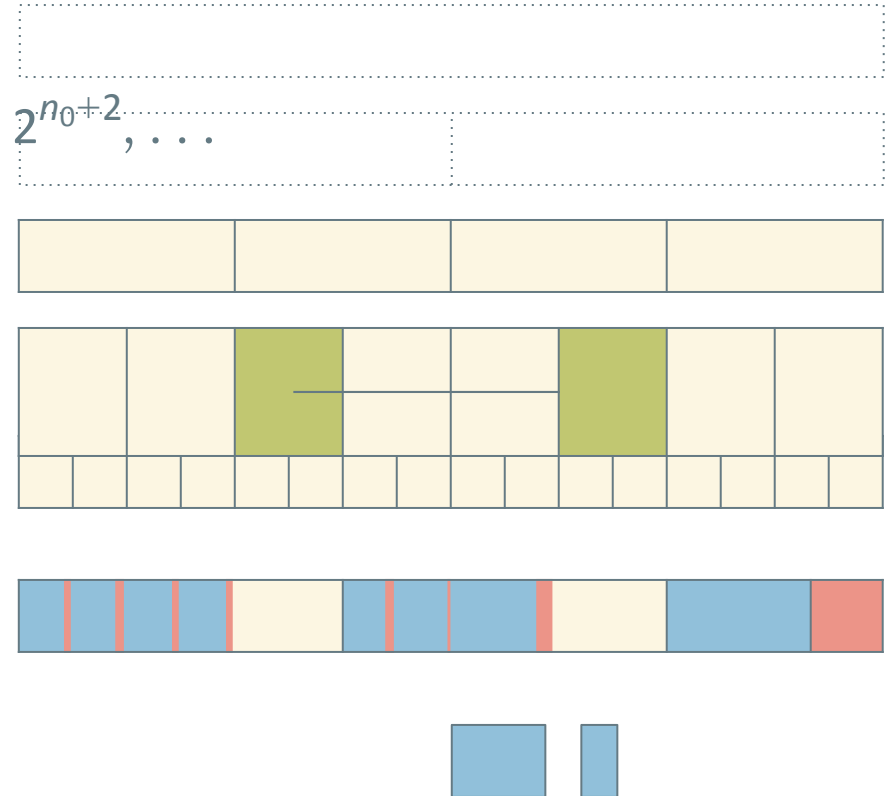


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

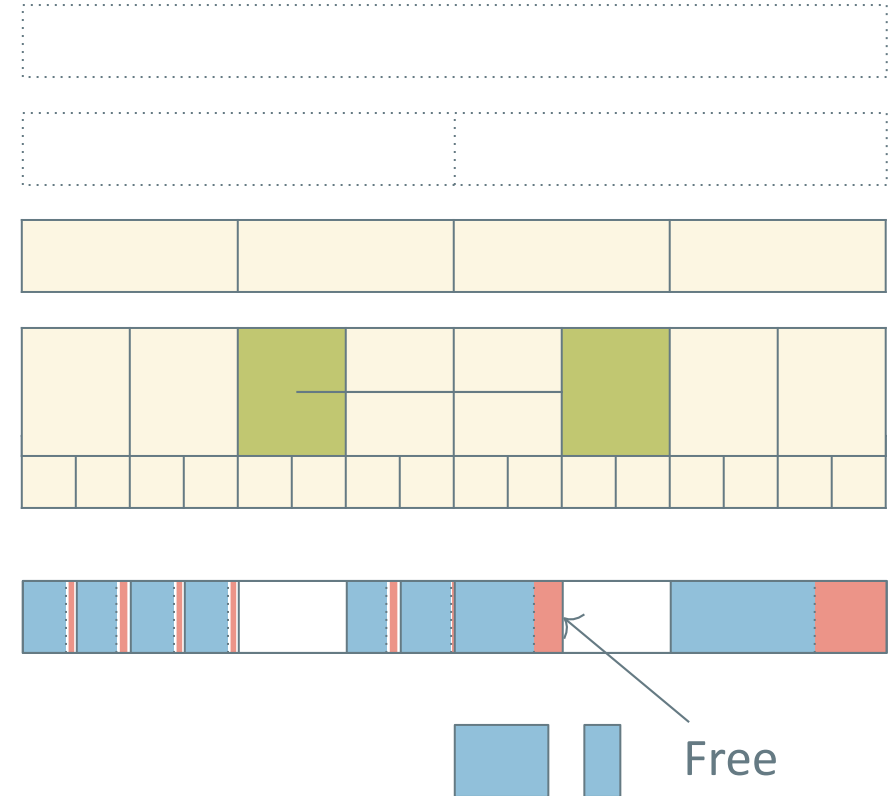


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

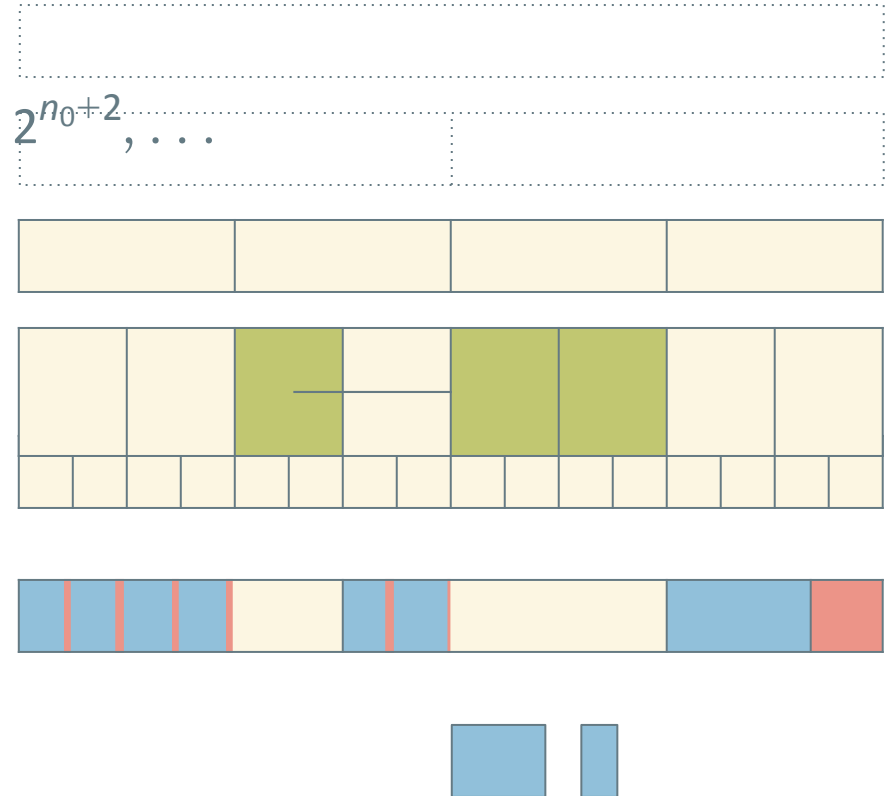


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

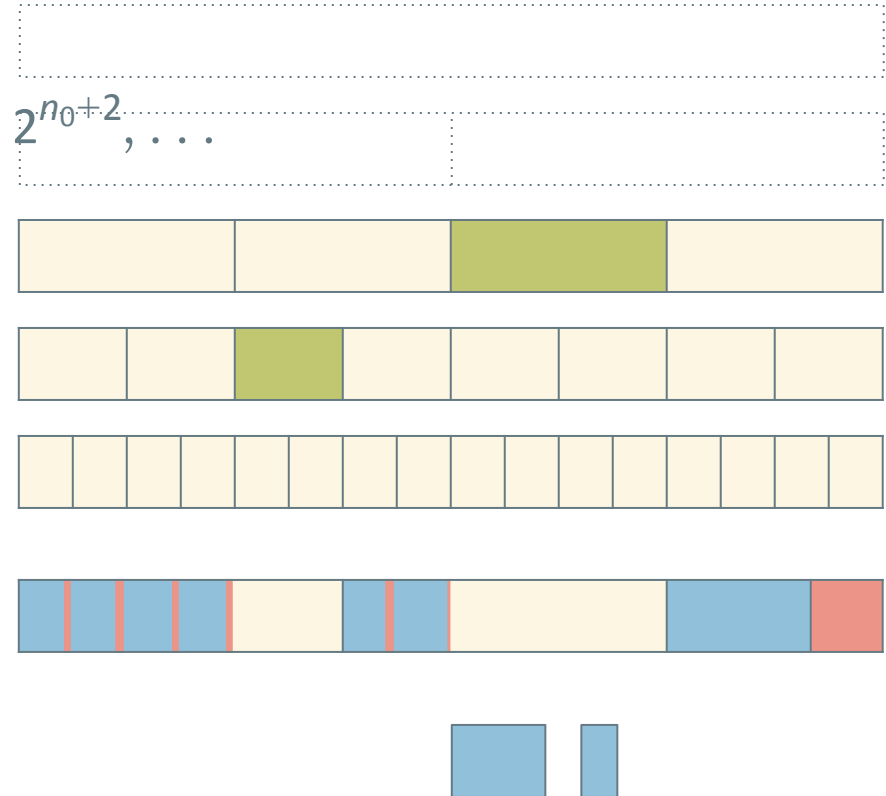


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

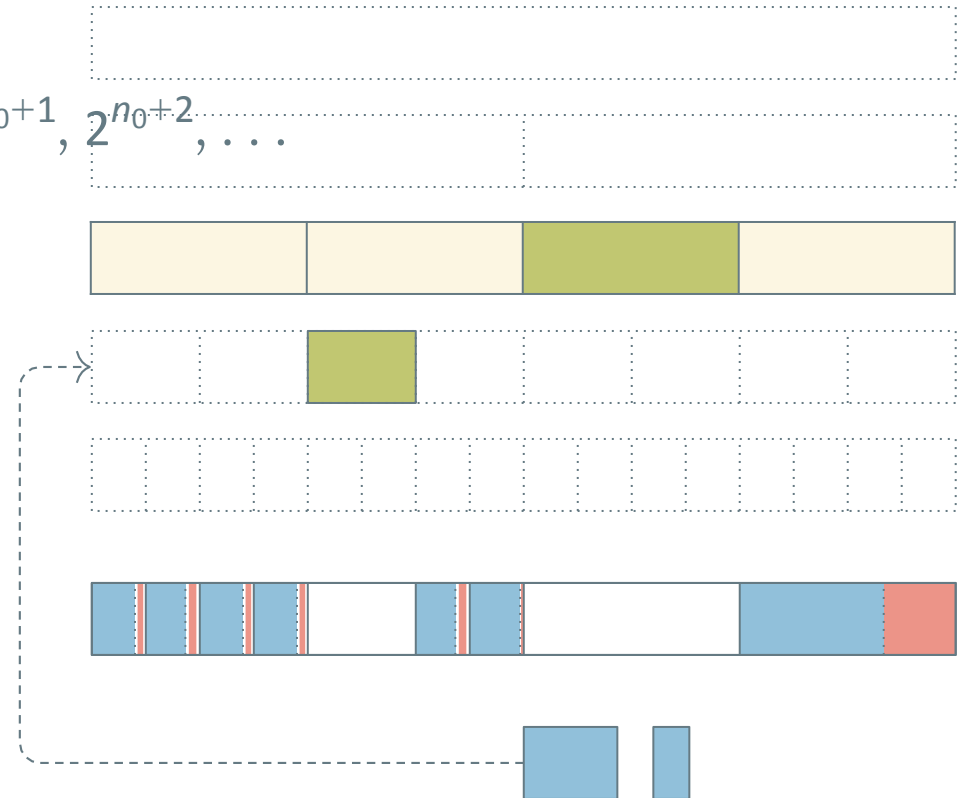


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

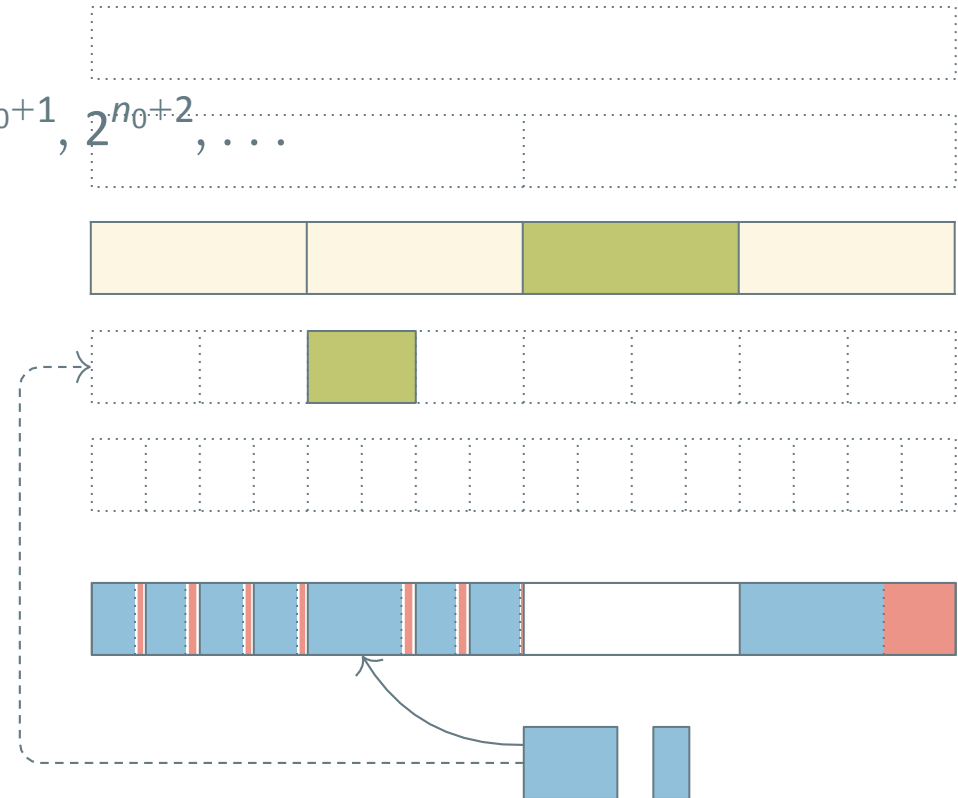


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

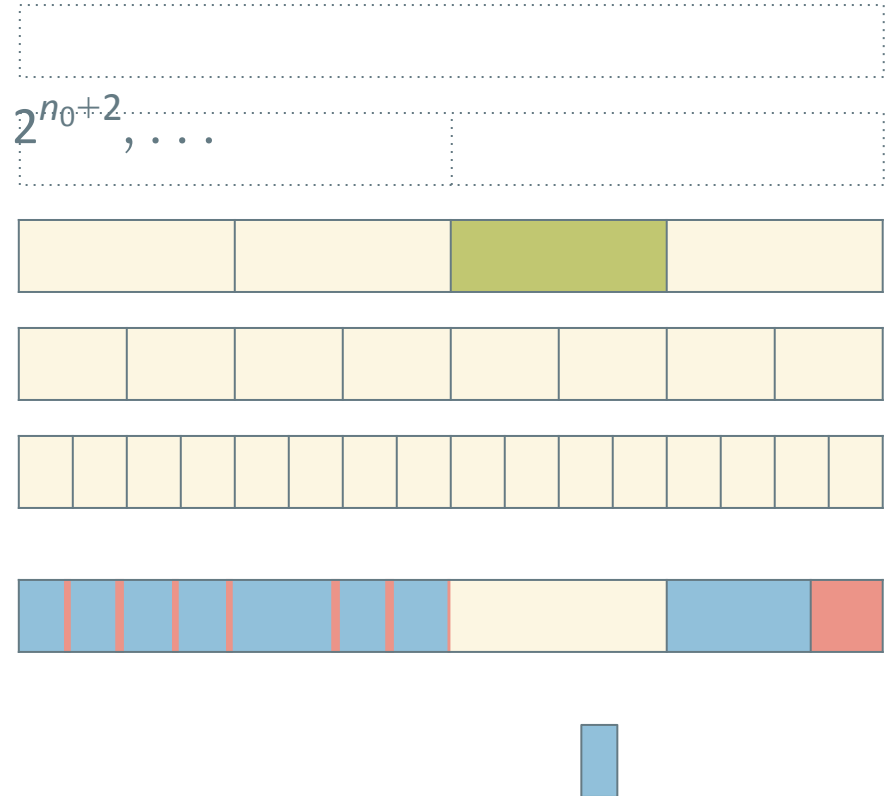


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

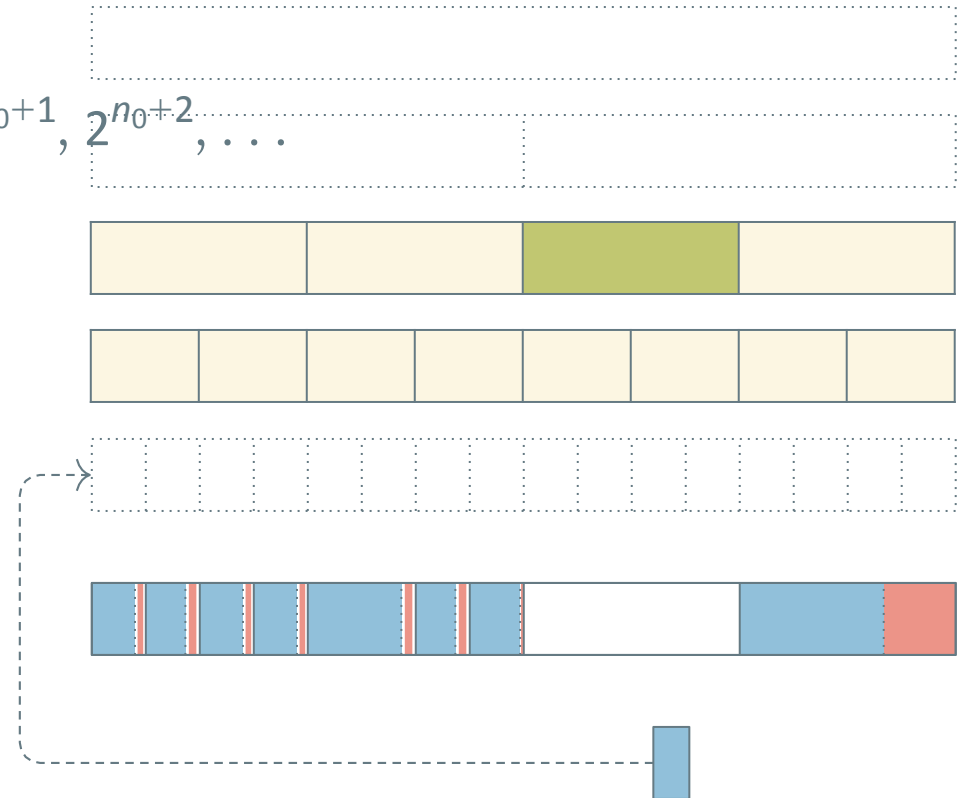


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

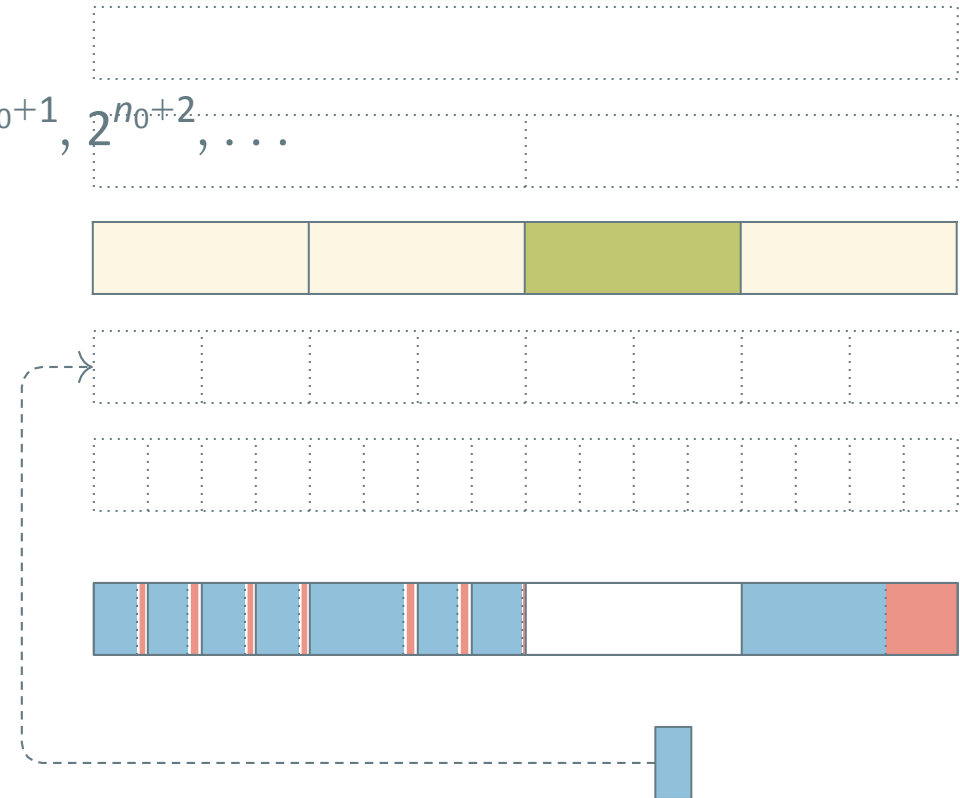


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

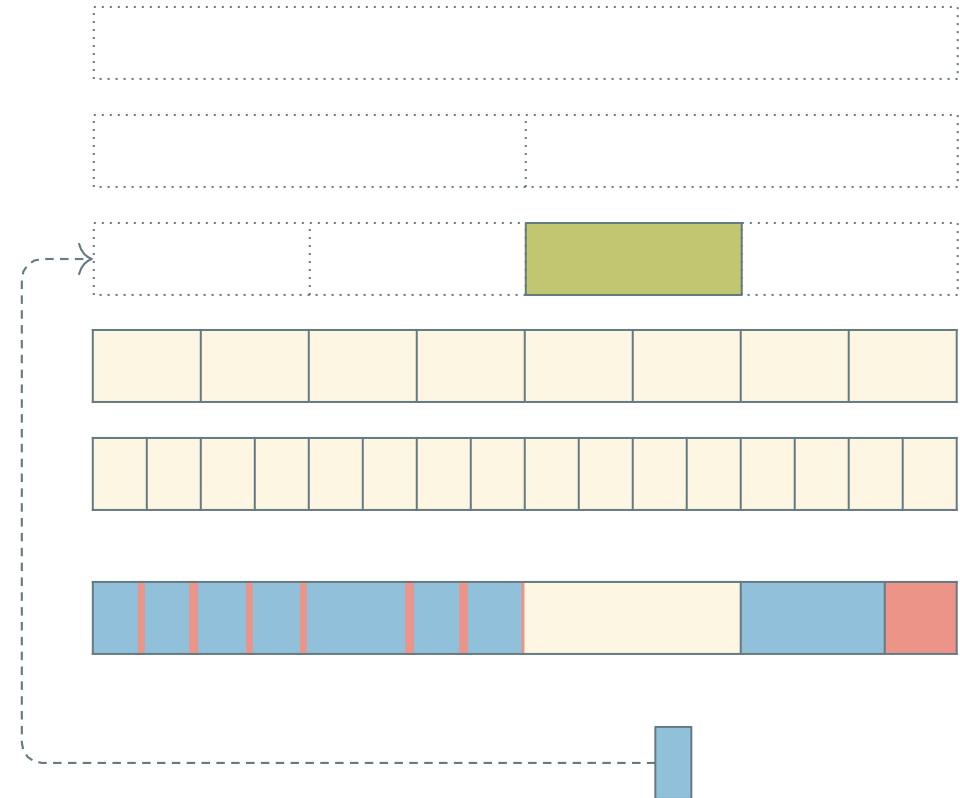


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

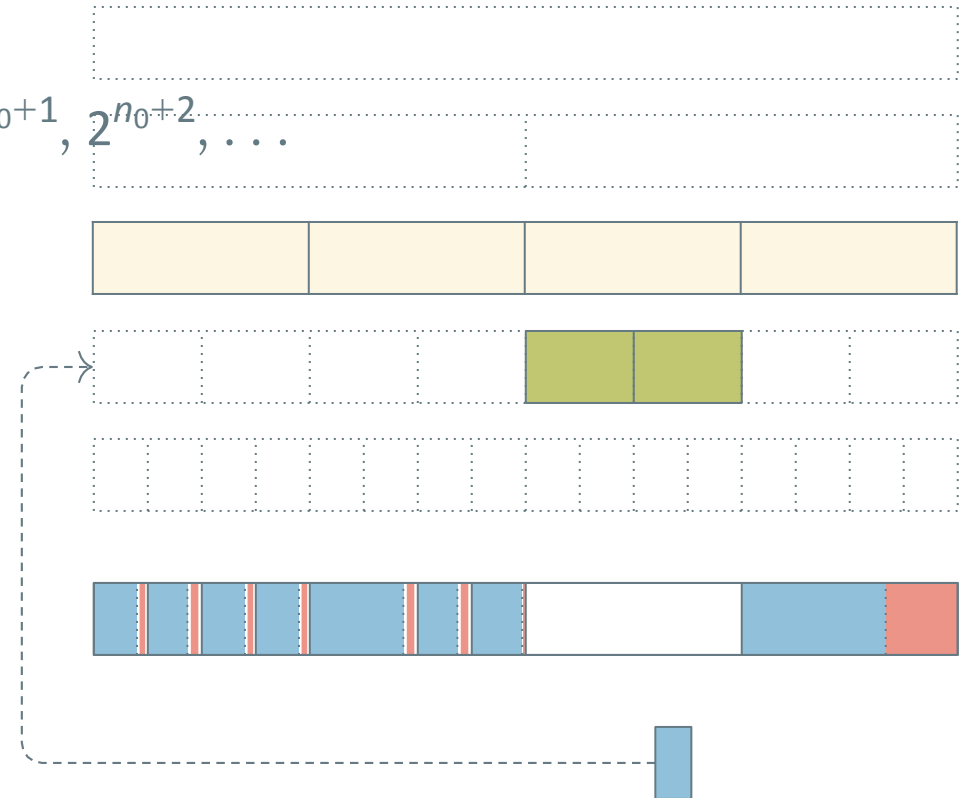


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

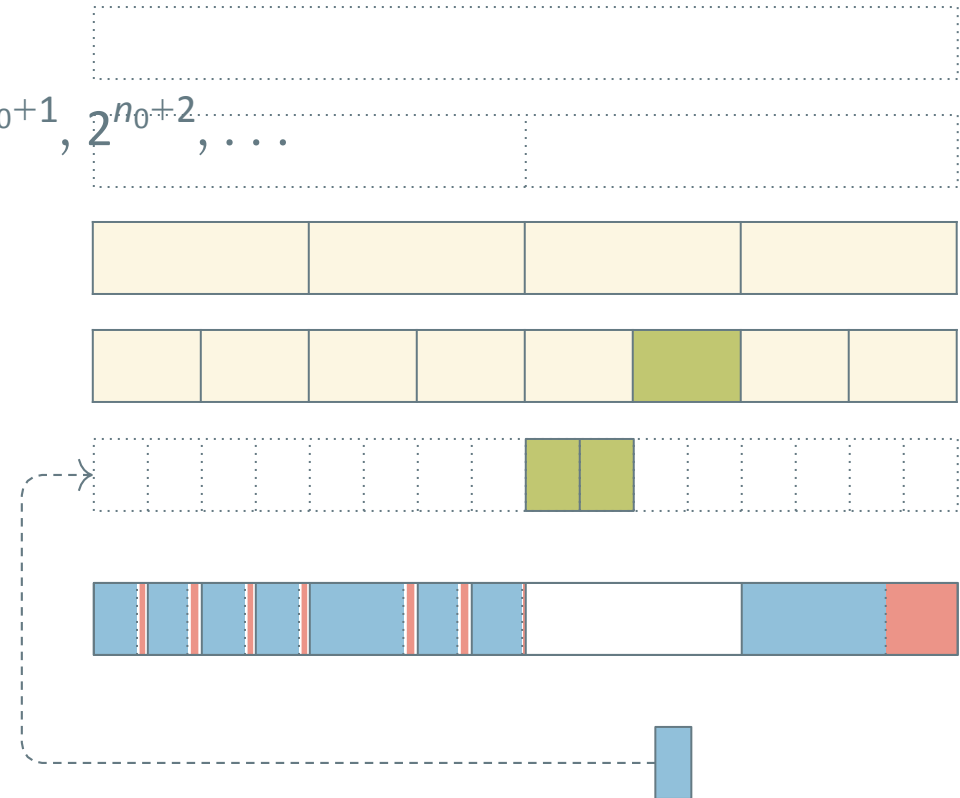


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

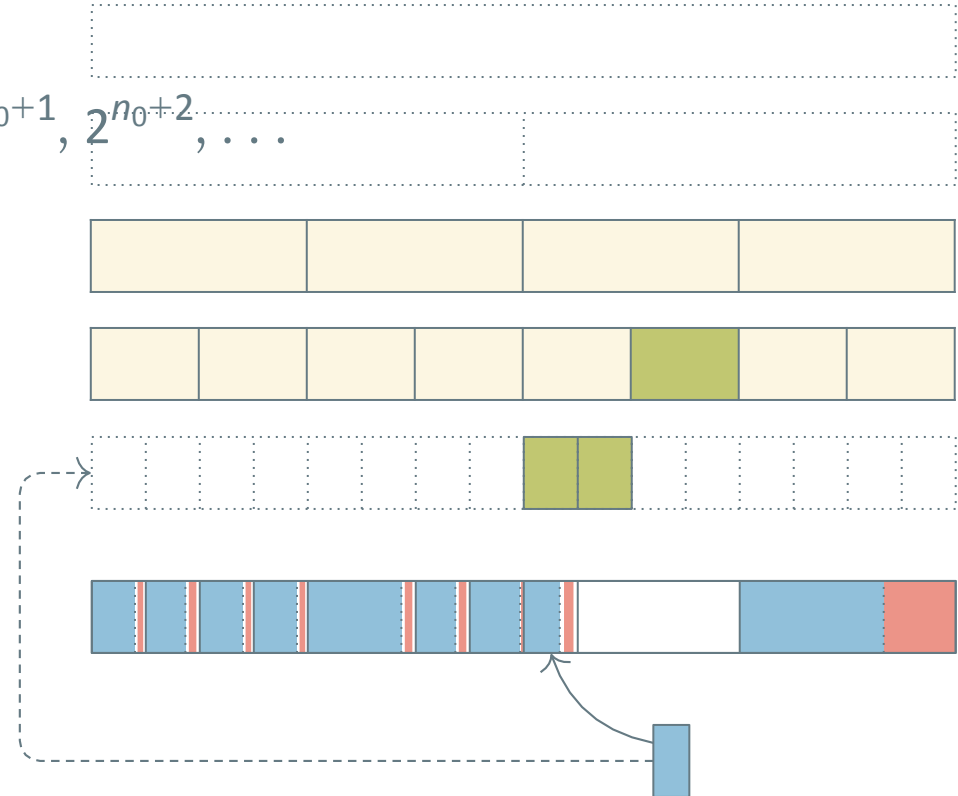


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size 2^n
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

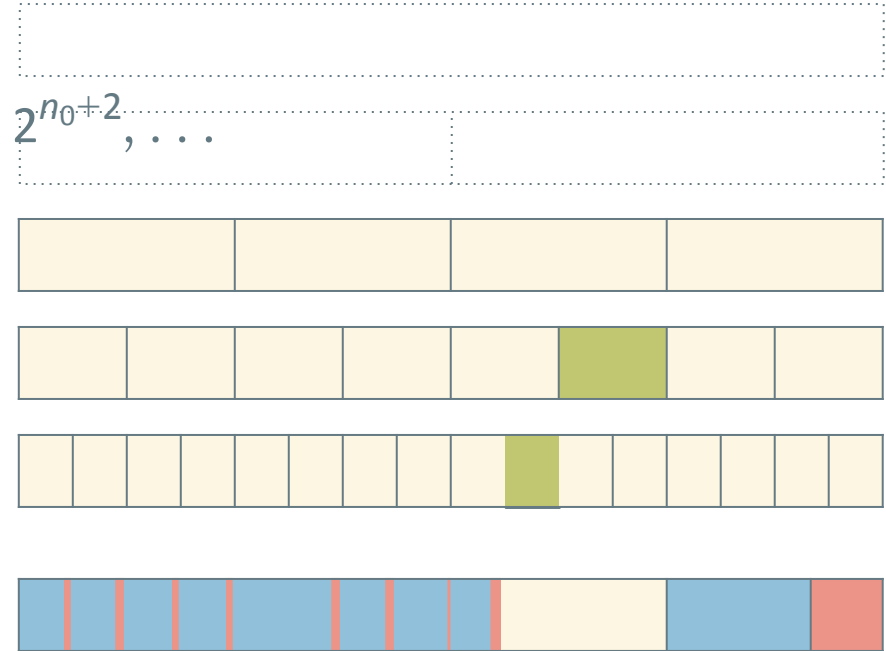


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$

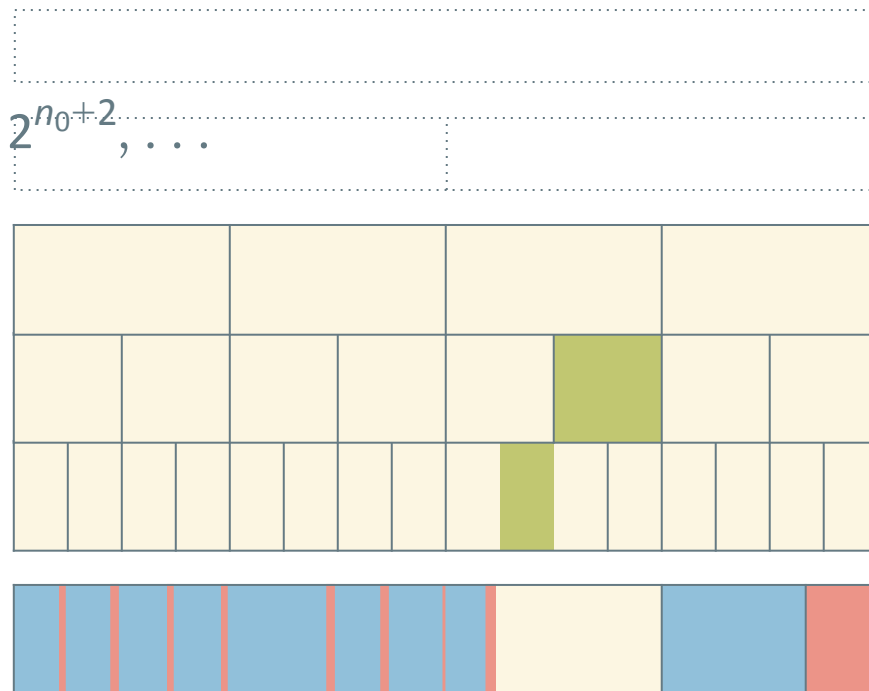


COST OF HEAP ALLOCATION

Single free list: Linear cost to find a block to accommodate each request.

Buddy system:

- Blocks of size $2^{n_0}, 2^{n_0+1}, 2^{n_0+2}, \dots$
- Separate free list for each block size
- If block of size 2^k is unavailable, split block of size 2^{k+1}
- If block of size 2^k is deallocated and its buddy is free, merge them
- Worst-case cost: $\log(\text{memory size})$



Fibonacci heap:

- Block sizes are Fibonacci numbers: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- Less fragmentation

HEAP DEALLOCATION

Explicit deallocation by programmer:

- Used in Pascal, C, C++, ...
- Efficient
- May lead to bugs that are difficult to find:
 - Dangling pointers/references from deallocating too soon.
 - Memory leaks from not deallocating.

HEAP DEALLOCATION

Explicit deallocation by programmer:

- Used in Pascal, C, C++, ...
- Efficient
- May lead to bugs that are difficult to find:
 - Dangling pointers/references from deallocating too soon.
 - Memory leaks from not deallocating.

Automatic deallocation by garbage collector:

- Using in Java, Python, declarative programming languages, ...
- Can add significant runtime overhead
- Safer

SCOPE

- The textual region of the program in which a binding is active is its scope.
- Informally, a scope is a program region of maximal size in which no bindings change (or at least none are destroyed). Typically, a scope is the body of a module, class, subroutine, or structured control flow statement, sometimes called a block. In C family languages it would be delimited with {...} braces.
- The scope of a binding is determined statically or dynamically.
 - In most modern languages, the scope of a binding is determined statically, that is, at compile time. In C, for example, we introduce a new scope upon entry to a subroutine. We create bindings for local objects and deactivate bindings for global objects that are “hidden” by local objects of the same name. On subroutine exit, we destroy bindings for local variables and reactivate bindings for any global objects that were hidden. These manipulations of bindings may at first glance appear to be run-time operations, but they do not require the execution of any code. We can look at a C program and know which names refer to which objects at which points in the program based on purely textual rules. For this reason, C is said to be statically scoped (lexically scoped). Other languages, including APL, Snobol, and early dialects of Lisp, are dynamically scoped: their bindings depend on the flow of execution at run time.

STATIC SCOPE

- With STATIC (LEXICAL) SCOPE RULES (e.g., C, Java and python), a scope is defined in terms of the physical (lexical) structure of the program:
 - The determination of scopes can be made by the compiler.
 - All bindings for identifiers can be resolved by examining the program. Typically, we choose the most recent, active binding made at compile time.
 - Static scoping (also called lexical scoping) rule examples:
 - one big scope (old Basic),
 - scope of a function (variables live through a function execution)
 - block scope (a local var. is available in the block in which is defined)
 - nested subroutines (have access to the variables defined in the parent)
 - if a variable is active in one or more scopes, then the closest nested scope rule applies
 - Most compiled languages, C, Pascal, Java and python included, employ static scope rules

STATIC SCOPE

- Nested Subroutines

- The ability to nest subroutines inside each other, introduced in Algol 60, is a feature of many modern languages, including Pascal, Ada, ML, Python etc.
- Nesting gives rise to the **closest nested scope rule** for bindings from names to objects: a name that is introduced in a declaration is known in the scope in which it is declared, and in each internally nested scope, unless it is hidden by another declaration of the same name in one or more nested scopes.
- To find the object corresponding to a given name, we look for a declaration with that name in the current, innermost scope. If there is one, it defines the active binding for the name. Otherwise, we look for a declaration in the immediately surrounding scope. We continue outward, examining successively surrounding scopes, until we reach the outer nesting level of the program, where global objects are declared. If no declaration is found at any level, then the program is in error

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );
  var X : real;
  procedure P2( A2 : T2 );
    procedure P3( A3 : T3 );
      begin
        ...
      end;
    begin
      ...
    end;
  procedure P4( A4 : T4 );
    function F1( A5 : T5 ) : T6;
      var X : integer;
      begin
        ...
      end;
    begin
      ...
    end;
  begin
    ...
  end;
```

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );
  var X : real;
  procedure P2( A2 : T2 );
    procedure P3( A3 : T3 );
      begin
        ...
      end;
    begin
      ...
    end;
  procedure P4( A4 : T4 );
    function F1( A5 : T5 ) : T6;
      var X : integer;
      begin
        ...
      end;
    begin
      ...
    end;
  begin
    ...
  end;
```

What's visible inside P1's body?

What's visible inside P2's body?

What's visible inside P3's body?

What's visible inside P4's body?

What's visible inside F1's body?

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );  
  var X : real;  
  procedure P2( A2 : T2 );  
    procedure P3( A3 : T3 );  
      begin  
        ...  
      end;  
    begin  
      ...  
    end;  
  procedure P4( A4 : T4 );  
    function F1( A5 : T5 ) : T6;  
      var X : integer;  
      begin  
        ...  
      end;  
    begin  
      ...  
    end;  
  begin  
    ...  
  end;
```

What's visible inside P1's body?

P1, A1, X₁, P2, P4

What's visible inside P2's body?

What's visible inside P3's body?

What's visible inside P4's body?

What's visible inside P5's body?

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );  
  var X : real;  
  procedure P2( A2 : T2 );  
    procedure P3( A3 : T3 );  
      begin  
        ...  
      end;  
    begin  
      ...  
    end;  
  procedure P4( A4 : T4 );  
    function F1( A5 : T5 ) : T6;  
      var X : integer;  
      begin  
        ...  
      end;  
    begin  
      ...  
    end;  
  begin  
    ...  
  end;
```

What's visible inside P1's body?

P1, A1, X₁, P2, P4

What's visible inside P2's body?

P1, A1, X₁, P2, P3, A2

What's visible inside P3's body?

What's visible inside P4's body?

What's visible inside P5's body?

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );  
  var X : real;  
  procedure P2( A2 : T2 );  
    procedure P3( A3 : T3 );  
      begin  
        ...  
      end;  
    begin  
      ...  
    end;  
  procedure P4( A4 : T4 );  
    function F1( A5 : T5 ) : T6;  
      var X : integer;  
      begin  
        ...  
      end;  
    begin  
      ...  
    end;  
  begin  
    ...  
  end;
```

What's visible inside P1's body?

P1, A1, X₁, P2, P4

What's visible inside P2's body?

P1, A1, X₁, P2, P3, A2

What's visible inside P3's body?

P1, A1, X₁, P2, P3, A2, A3

What's visible inside P4's body?

What's visible inside F1's body?

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );
  var X : real;
  procedure P2( A2 : T2 );
    procedure P3( A3 : T3 );
    begin
      ...
    end;
  begin
    ...
  end;
  procedure P4( A4 : T4 );
    function F1( A5 : T5 ) : T6;
    var X : integer;
    begin
      ...
    end;
  begin
    ...
  end;
begin
  ...
end;
```

What's visible inside P1's body?

P1, A1, X₁, P2, P4

What's visible inside P2's body?

P1, A1, X₁, P2, P3, A2

What's visible inside P3's body?

P1, A1, X₁, P2, P3, A2, A3

What's visible inside P4's body?

P1, A1, X₁, P2, P4, A4, F1

What's visible inside F1's body?

LEXICAL SCOPING IN PASCAL

```
procedure P1( A1 : T1 );
  var X : real;
  procedure P2( A2 : T2 );
    procedure P3( A3 : T3 );
    begin
      ...
    end;
  begin
    ...
  end;
  procedure P4( A4 : T4 );
    function F1( A5 : T5 ) : T6;
    var X : integer;
    begin
      ...
    end;
  begin
    ...
  end;
begin
  ...
end;
```

What's visible inside P1's body?

P1, A1, X₁, P2, P4

What's visible inside P2's body?

P1, A1, X₁, P2, P3, A2

What's visible inside P3's body?

P1, A1, X₁, P2, P3, A2, A3

What's visible inside P4's body?

P1, A1, X₁, P2, P4, A4, F1

What's visible inside F1's body?

P1, A1, P2, P4, A4, F1, A5, X₂

STATIC SCOPE

- **Access to Nonlocal Objects**

- As we already seen that the compiler can arrange for a frame pointer register to point to the frame of the currently executing subroutine at run time. Due to this, the object when not defend in current subroutine but accessed by subroutine, then how to resolve it?
- The simplest way in which to find the frames of surrounding scopes is to maintain a **static link** in each frame that points to the “parent” frame: the frame of the most recent invocation of the lexically surrounding subroutine. If a subroutine is declared at the outermost nesting level of the program, then its frame will have a null static link at run time.

STATIC SCOPE

- Access to Nonlocal Objects

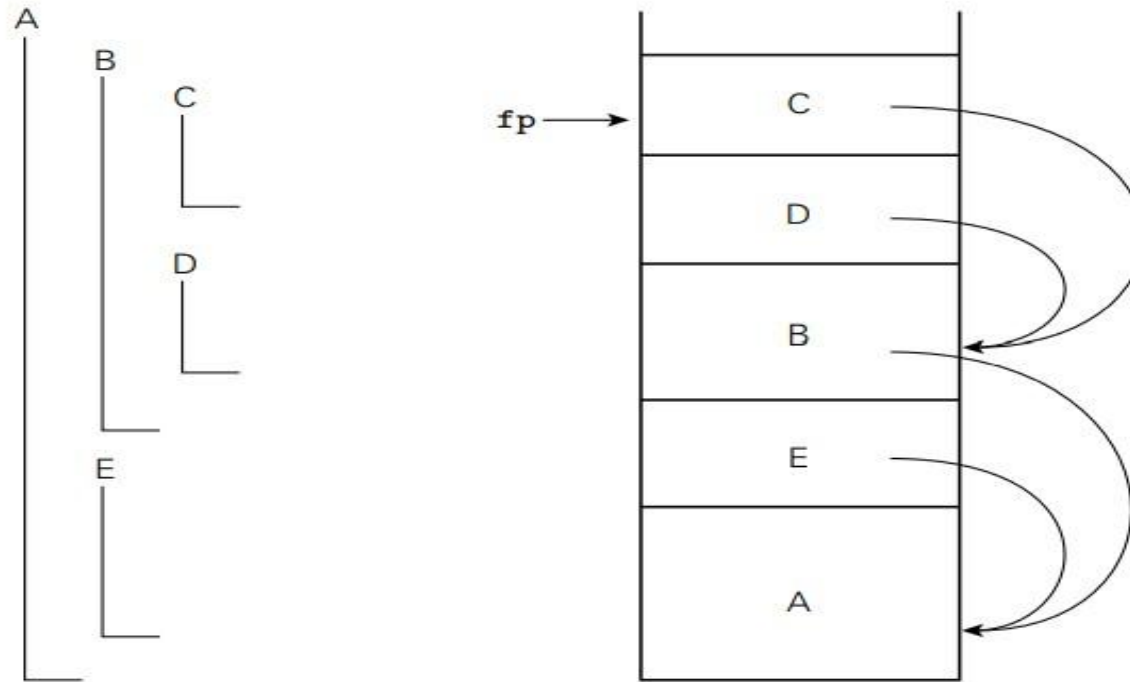
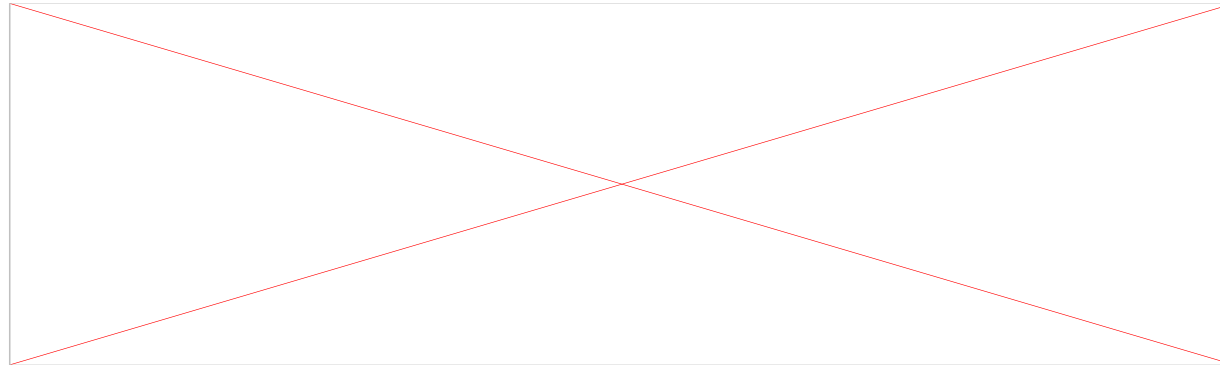


Figure 3.5 Static chains. Subroutines A, B, C, D, and E are nested as shown on the left. If the sequence of nested calls at run time is A, E, B, D, and C, then the static links in the stack will look as shown on the right. The code for subroutine C can find local objects at known offsets from the frame pointer: It can find local objects of the surrounding scope, B, by dereferencing its static chain once and then applying an offset. It can find local objects in B's surrounding scope, A, by dereferencing its static chain twice and then applying an offset.

STATIC SCOPE

- **Declaration Order**

- Suppose an object *x* is declared somewhere within block *B*. Does the scope of *x* include the portion of *B* before the declaration, and if so can *x* actually be used in that portion of the code?
- Several early languages, including Algol 60 and Lisp, required that all declarations appear at the beginning of their scope.
- Pascal modified the requirement to say that names must be declared before they are used. At the same time, however, Pascal retained the notion that the scope of a declaration is the entire surrounding block. These two rules can interact in surprising ways:



Pascal says that the second declaration of *N* covers all of *foo*, so the semantic analyzer should complain on line 5 that *N* is being used before its declaration.

STATIC SCOPE

- **Declaration Order**

- To avoid this complication, most Pascal successors (like C, and some dialects of Pascal itself) specify that the scope of an identifier is not the entire block in which it is declared (excluding holes), but rather the portion of that block from the declaration to the end (again excluding holes).

```
#include <stdio.h>
int a=20;
void foo()
{
    printf("value of a =%d\n",a);
    int a=10;
    a=2*a;
    printf("value of a =%d\n",a);
}
int main()
{
    a=11;
    foo();
    printf("value of a=%d",a);
    return 0;
}
```

```
a=20
def foo():
    print("a value is ",a)
    a=10
    a=a*2
    print("a value is ",a)
a=11
foo()
print("a value is ",a)
```

STATIC SCOPE

- **Declaration Order**

- To avoid this complication, most Pascal successors (and some dialects of Pascal itself) specify that the scope of an identifier is not the entire block in which it is declared (excluding holes), but rather the portion of that block from the declaration to the end (again excluding holes).

```
#include <stdio.h>
int a=20;
void foo()
{
    printf("value of a =%d\n",a);
    int a=10;
    a=2*a;
    printf("value of a =%d\n",a);
}
int main()
{
    a=11;
    foo();
    printf("value of a=%d",a);
    return 0;
}
```

C Code:
11, 20, 11

```
a=20
def foo():
    print("a value is ",a)
    a=10
    a=a*2
    print("a value is ",a)
a=11
foo()
print("a value is ",a)
```

STATIC SCOPE

- **Declaration Order**

- To avoid this complication, most Pascal successors (and some dialects of Pascal itself) specify that the scope of an identifier is not the entire block in which it is declared (excluding holes), but rather the portion of that block from the declaration to the end (again excluding holes).

```
#include <stdio.h>
int a=20;
void foo()
{
    printf("value of a =%d\n",a);
    int a=10;
    a=2*a;
    printf("value of a =%d\n",a);
}
int main()
{
    a=11;
    foo();
    printf("value of a=%d",a);
    return 0;
}
```

C Code:
11, 20, 11
Python code:
Error,
Name used before declaration.

```
a=20
def foo():
    print("a value is ",a)
    a=10
    a=a*2
    print("a value is ",a)
a=11
foo()
print("a value is ",a)
```

STATIC SCOPE

- **Declaration Order**

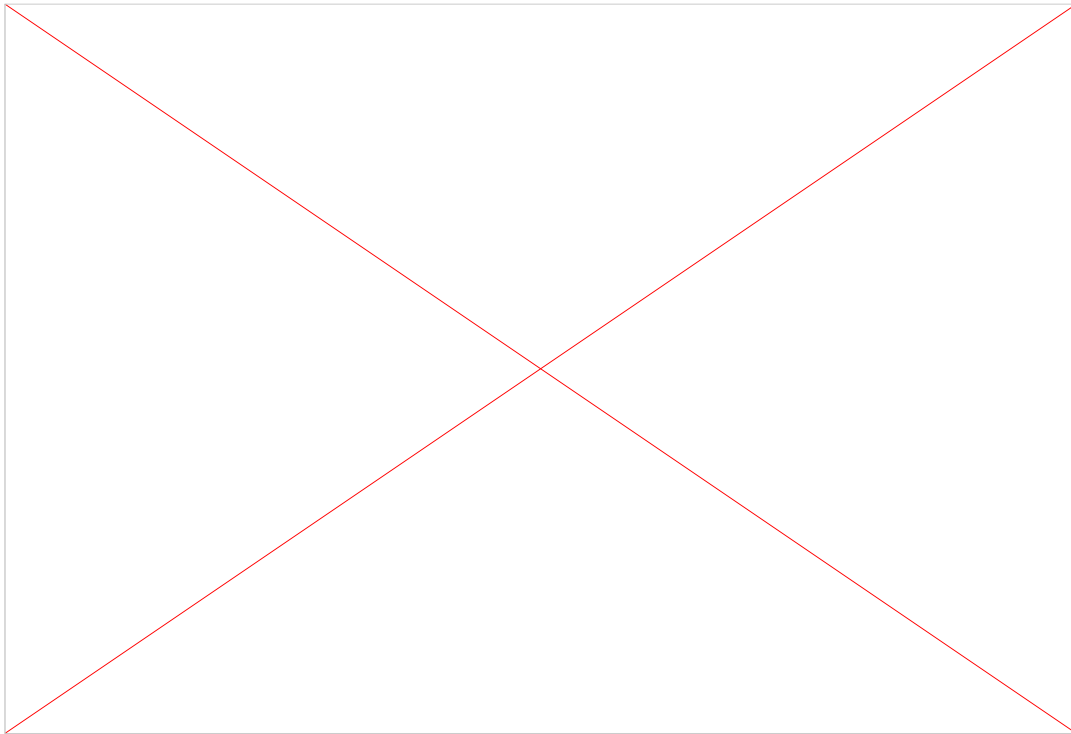
- Recursive types and subroutines introduce a problem for languages that require names to be declared before they can be used
- A declaration introduces a name , but may omit certain implementation details.
- A definition describes the object in sufficient detail for the compiler to determine its implementation. If a declaration is not complete enough, then a separate definition must appear somewhere else in the scope. In C we can write:

```
struct manager;                                /* declaration only */
struct employee {
    struct manager *boss;
    struct employee *next_employee;
    ...
};
struct manager {                                /* definition */
    struct employee *first_employee;
    ...
};
```

The initial declaration of manager needed only to introduce a name: since pointers are all the same size, the compiler could determine the implementation of employee without knowing any manager details.

DYNAMIC SCOPE

- In a language with dynamic scoping, the bindings between names and objects depend on the flow of control at run time, and in particular on the order in which subroutines are called.
- In comparison to the static scope rules, dynamic scope rules are generally quite simple: the “current” binding for a given name is the one encountered most recently during execution, and not yet destroyed by returning from its scope.



- If static scoping is in effect, this program prints a 1.
- If dynamic scoping is in effect, the output depends on the value read at line 8 at run time: if the input is positive, the program prints a 2; otherwise it prints a 1.
- Static scope rules require that the reference resolve to the closest lexically enclosing declaration, namely the global n.
- Dynamic scope rules, on the other hand, require that we choose the most recent, active binding for n at run time.

Binding of reference environment

- Referencing environment:
 - At any given point in a program's execution, the set of active bindings is called the current referencing environment. The set is principally determined by static or dynamic scope rules.
- Shallow binding :
 - Free variables are bound when the subroutine is called. Or binds the environment at the time the procedure is actually called.
- Deep binding:
 - Free variables are bound when the subroutine is first passed as a parameter. Or binds the environment at the time the procedure is passed as an argument.
- This is important using both static and dynamic scoping and is known as the funarg problem.

Example: static vs. dynamic shallow vs. dynamic deep

```
var total = 0
```

```
def a():  
    total += 1
```

```
def b(F):  
    var total = 0  
    F()  
    print "B", total  
def c():  
    var total = 0  
    b(a)  
    print "C", total  
c()  
print "T", total
```

	static	dynamic shallow	dynamic deep
B	0	1	0
C	0	0	1
T	1	0	0

Example: static vs. dynamic shallow vs. dynamic deep

```
x: integer := 1  
y: integer := 2
```

```
procedure add
```

```
  x := x + y
```

```
procedure second(P:procedure)
```

```
  x:integer := 2
```

```
  P()
```

```
procedure first
```

```
  y:integer := 3
```

```
  second(add)
```

```
first()
```

```
write_integer(x)
```

static scoping would print 3.

- shallow binding just traverses up until it finds the nearest variable that corresponds to the name (increments the local x), so the answer would be 1.

- dynamic scoping with deep binding: when add is passed into second the environment of add is $x = 1$, $y = 3$ and the x is the global x so it writes 4 into the global x, which is the one picked up by the write_integer.

Example: static vs. dynamic shallow vs. dynamic deep

```
int x = 10;

function f( int a )
{
    x = x + a;
}

function g( function h )
{
    int x = 30;
    h( 100 );
    print( x );
}

function main()
{
    g( f );
    print( x );
}
```

What is the output of this program using deep binding?

What is the output of this program using shallow binding?

Example: static vs. dynamic shallow vs. dynamic deep

```
int x = 10;

function f( int a )
{
    x = x + a;
}

function g( function h )
{
    int x = 30;
    h( 100 );
    print( x );
}

function main()
{
    g( f );
    print( x );
}
```

What is the output of this program using deep binding?

30
110

What is the output of this program using shallow binding?

Example: static vs. dynamic shallow vs. dynamic deep

```
int x = 10;

function f( int a )
{
    x = x + a;
}

function g( function h )
{
    int x = 30;
    h( 100 );
    print( x );
}

function main()
{
    g( f );
    print( x );
}
```

What is the output of this program using deep binding?

30
110

What is the output of this program using shallow binding?

130
10

ALIASING AND OVERLOADING

- So far we have assumed that there is a one-to-one mapping between names and visible objects in any given point in a program.
- Two or more names that refer to the same object at the same point in the program are said to be **aliases**.
- A name that can refer to more than one object at a given point in the program is said to be **overloaded**.

```
double sum, sum_of_squares;
...
void accumulate(double& x)    // x is passed by reference
{
    sum += x;
    sum_of_squares += x * x;
}
...
accumulate(sum);
```

If `sum` is passed as an argument to `accumulate`, then `sum` and `x` will be aliases for one another, and the program will probably not do what the programmer intended.

ALIASING AND OVERLOADING

- As a general rule, aliases tend to make programs more confusing.
- They also make it much more difficult for a compiler to perform certain important code improvements.

```
int a, b, *p, *q;  
...  
a = *p;      /* read from the variable referred to by p */  
*q = 3;      /* assign to the variable referred to by q */  
b = *p;      /* read from the variable referred to by p */
```

The initial assignment to “a” will require that *p be loaded into a register. Since accessing memory is expensive, the compiler will want to hang on to the loaded value and reuse it in the assignment to b. But it can not do unless it can verify that p and q refer to the same object.

